

# 编译原理课程大作业报告提纲

类 Rust 语言词法分析与语法分析工具设计与实现

学院：计算机科学与技术学院

专业：计算机科学与技术

小组成员：白佳煜、胡桥健、庄子硕

学号：2352733、2351295、2350998

指导教师：卫志华、高珍

2026 年 5 月 16 日

# 目录

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>实验背景与任务要求</b>                      | <b>6</b>  |
| 1.1      | 实验背景 . . . . .                        | 6         |
| 1.2      | 任务目标 . . . . .                        | 6         |
| 1.3      | 任务范围与实现边界 . . . . .                   | 7         |
| <b>2</b> | <b>系统总体设计</b>                         | <b>7</b>  |
| 2.1      | 总体设计思路 . . . . .                      | 7         |
| 2.2      | 系统模块划分 . . . . .                      | 8         |
| 2.3      | 数据流设计 . . . . .                       | 9         |
| <b>3</b> | <b>词法分析器设计与实现</b>                     | <b>10</b> |
| 3.1      | 词法分析器的功能定位 . . . . .                  | 10        |
| 3.2      | 词法分析状态转换图 . . . . .                   | 11        |
| 3.3      | 核心数据结构设计 . . . . .                    | 12        |
| 3.3.1    | 位置信息 Position . . . . .               | 12        |
| 3.3.2    | Token 结构 . . . . .                    | 12        |
| 3.3.3    | TokenKind 枚举 . . . . .                | 12        |
| 3.3.4    | Keyword 枚举 . . . . .                  | 13        |
| 3.3.5    | 词法错误 LexError 与结果 LexResult . . . . . | 14        |
| 3.4      | 词法分析器的内部状态设计 . . . . .                | 14        |
| 3.5      | 公共接口设计 . . . . .                      | 15        |
| 3.6      | 主扫描流程设计 . . . . .                     | 15        |
| 3.7      | 标识符与关键字识别 . . . . .                   | 16        |
| 3.8      | 整数识别 . . . . .                        | 17        |
| 3.9      | 复合 Token 与最长匹配策略 . . . . .            | 17        |
| 3.10     | 单字符 Token 识别 . . . . .                | 18        |
| 3.11     | 空白字符与注释处理 . . . . .                   | 18        |
| 3.11.1   | 空白字符处理 . . . . .                      | 18        |
| 3.11.2   | 单行注释处理 . . . . .                      | 19        |
| 3.11.3   | 块注释处理 . . . . .                       | 19        |
| 3.12     | 字符读取与位置维护 . . . . .                   | 19        |
| 3.13     | Token 输出与格式化 . . . . .                | 20        |
| 3.14     | 错误处理机制 . . . . .                      | 21        |
| 3.15     | 词法分析器实现特点 . . . . .                   | 21        |

|                               |           |
|-------------------------------|-----------|
| <b>4 语法分析器设计与实现</b>           | <b>22</b> |
| 4.1 语法分析器的功能定位                | 22        |
| 4.2 语法分析器公共接口设计               | 24        |
| 4.3 语法错误表示                    | 24        |
| 4.4 AST 总体结构设计                | 25        |
| 4.5 Program 与 FunctionDecl 设计 | 26        |
| 4.6 Binding 与类型节点设计           | 26        |
| 4.6.1 Binding 节点              | 26        |
| 4.6.2 TypeNode 节点             | 27        |
| 4.7 语句节点设计                    | 28        |
| 4.8 表达式节点设计                   | 29        |
| 4.9 Parser 内部状态设计             | 30        |
| 4.10 程序与函数声明解析                | 30        |
| 4.10.1 程序解析                   | 30        |
| 4.10.2 函数声明解析                 | 31        |
| 4.11 参数与变量绑定解析                | 32        |
| 4.12 类型解析                     | 33        |
| 4.13 语句块与尾表达式解析               | 34        |
| 4.14 语句解析                     | 34        |
| 4.14.1 空语句                    | 35        |
| 4.14.2 变量声明语句                 | 35        |
| 4.14.3 返回语句                   | 35        |
| 4.14.4 while 循环               | 35        |
| 4.14.5 for 循环                 | 35        |
| 4.14.6 break 与 continue       | 36        |
| 4.15 赋值语句解析与左值检查              | 36        |
| 4.16 表达式解析与优先级处理              | 37        |
| 4.16.1 比较表达式                  | 37        |
| 4.16.2 加减表达式                  | 38        |
| 4.16.3 乘除表达式                  | 38        |
| 4.16.4 一元表达式                  | 38        |
| 4.17 后缀表达式解析                  | 39        |
| 4.18 基本表达式解析                  | 40        |
| 4.18.1 数字与标识符                 | 40        |
| 4.18.2 块表达式                   | 40        |

|                      |           |
|----------------------|-----------|
| 目录                   | 4         |
| 4.18.3 if 表达式        | 40        |
| 4.18.4 loop 表达式      | 41        |
| 4.18.5 数组字面量         | 41        |
| 4.18.6 元组与括号表达式      | 41        |
| 4.19 token 匹配辅助机制    | 41        |
| 4.20 错误报告机制          | 42        |
| 4.21 语法分析器实现特点       | 43        |
| <b>5 Web 可视化系统设计</b> | <b>43</b> |
| 5.1 设计目标             | 43        |
| 5.2 系统结构             | 44        |
| 5.3 服务端路由设计          | 44        |
| 5.4 请求与响应数据设计        | 45        |
| 5.5 Token 展示设计       | 46        |
| 5.6 AST 展示设计         | 47        |
| 5.7 错误信息展示设计         | 47        |
| 5.8 Web 可视化系统的作用     | 49        |
| <b>6 实验测试与结果分析</b>   | <b>49</b> |
| 6.1 测试环境             | 49        |
| 6.2 词法分析测试           | 49        |
| 6.2.1 关键字后缀测试        | 50        |
| 6.2.2 关键字、赋值号和整数拆分测试 | 50        |
| 6.2.3 注释跳过测试         | 50        |
| 6.2.4 最长匹配测试         | 51        |
| 6.2.5 未闭合块注释测试       | 52        |
| 6.2.6 词法分析结果         | 52        |
| 6.3 语法分析测试           | 52        |
| 6.3.1 最低语法规则测试       | 53        |
| 6.3.2 扩展语法测试         | 53        |
| 6.3.3 赋值目标错误测试       | 54        |
| 6.3.4 结束符测试          | 54        |
| 6.3.5 语法分析结果         | 56        |
| <b>7 改进方向</b>        | <b>56</b> |
| 7.1 增加符号表            | 56        |
| 7.2 实现基础类型检查         | 57        |

|                       |    |
|-----------------------|----|
| 目录                    | 5  |
| 7.3 实现可变性检查 . . . . . | 57 |
| 8 总结与展望               | 58 |

## 1 实验背景与任务要求

### 1.1 实验背景

编译器前端通常包括词法分析、语法分析和语义分析等阶段。其中，词法分析负责将源程序的字符序列转换为具有明确类别的 token 序列；语法分析则在 token 序列的基础上，根据语言文法识别程序结构，并构造抽象语法树。二者共同构成理解源程序结构的基础，也是后续语义检查、中间代码生成和优化工作的前提。

本次课程大作业以类 Rust 语言为对象，要求实现一个能够进行词法分析和语法分析的编译前端原型。Rust 语言具有较为清晰的语法结构，同时包含函数声明、变量绑定、可变性标记、表达式、控制流、引用、数组和元组等语言特性，因此适合作为编译原理实验中的分析对象。通过实现类 Rust 语言的词法分析器和语法分析器，可以将课堂中关于正规文法、有限自动机、上下文无关文法、语法树和递归下降分析等理论知识落实到具体程序中。

本项目的核心任务并不是实现完整的 Rust 编译器，而是在课程给定的类 Rust 语法范围内，完成从源代码输入到 token 序列输出、再到 AST 构造与展示的基本流程。项目最终应能够接收一段类 Rust 源程序，识别其中的词法单元，判断其是否符合基本语法规则，并以命令行或 Web 页面形式展示分析结果。

### 1.2 任务目标

结合课程要求和本项目的实际实现，本实验的目标可以概括为以下三个层次。

首先，在词法分析层面，需要设计一套能够覆盖类 Rust 词法规则的 token 类型体系，并实现相应的扫描逻辑。词法分析器应能够识别关键字、标识符、整数、运算符、界符、分隔符、特殊符号、注释和结束符等基本词法单元。同时，词法分析器还需要正确处理关键字与标识符之间的边界问题，例如 `if123` 应被识别为一个标识符，而 `if=123` 应被拆分为关键字 `if`、赋值号 `=` 和整数 `123`。

其次，在语法分析层面，需要根据课程给定的类 Rust 语法规则，实现对程序结构的识别与抽象语法树构造。语法分析器至少应支持函数声明、形参列表、返回语句、变量声明、赋值语句、基础表达式、比较运算、加减乘除运算、函数调用、if 选择结构和 while 循环结构等核心语法。由于这些结构基本覆盖了一个小型命令式语言的主要组成部分，因此它们也是本项目语法分析器设计的重点。

最后，在工程展示层面，项目需要提供清晰的分析结果输出方式。除命令行运行外，本项目还实现了本地 Web 页面，用于展示 token 表、AST 树和错误信息。这样既便于调试，也便于演示时直观展示词法分析和语法分析的过程与结果。

### 1.3 任务范围与实现边界

需要说明的是，本次大作业的重点在于词法分析和语法分析，而不是完整的语义分析。因此，本项目主要判断源程序在词法和语法层面是否符合规则，并构造相应的 AST；对于变量是否已经声明、赋值类型是否匹配、不可变变量是否被重新赋值、函数返回类型是否一致等问题，目前只在有限范围内处理，尚未形成完整的语义检查体系。

在最低要求之外，本项目还对部分类 Rust 扩展语法进行了支持，例如 `else`、`else if`、`for`、`loop`、`break`、`continue`、引用、数组、元组和表达式块等。这些扩展内容能够增强分析器对类 Rust 程序的覆盖能力，但其语义正确性检查并非本阶段的重点。

因此，本文后续将围绕以下问题展开：

1. 如何设计词法单元类型，并实现对类 Rust 源程序的逐字符扫描；
2. 如何处理关键字边界、最长匹配、注释跳过和错误定位等词法分析问题；
3. 如何设计 AST 节点结构，使其能够表达函数、语句、表达式和类型等语法结构；
4. 如何使用递归下降方法实现语法分析，并正确处理表达式优先级；
5. 如何通过测试样例和 Web 页面验证分析器的正确性与可展示性。

## 2 系统总体设计

### 2.1 总体设计思路

本项目面向类 Rust 语言的词法分析与语法分析任务，整体设计遵循编译器前端的基本处理流程：首先由词法分析器将源程序字符流转换为 token 序列，然后由语法分析器根据语法规则对 token 序列进行结构化分析，并构造抽象语法树，最后通过命令行或 Web 页面将分析结果展示给用户。

从系统职责上看，本项目并不试图实现完整的 Rust 编译器，而是聚焦于编译前端中最基础、最核心的两个阶段：词法分析和语法分析。词法分析阶段关注“源代码中有哪些词法单元”，语法分析阶段关注“这些词法单元能否组成合法的程序结构”。因此，系统总体设计的核心目标是建立一条清晰的分析流水线，使源程序能够依次经过输入、词法扫描、token 输出、语法解析、AST 构造和结果展示等步骤。

项目采用模块化设计，将词法分析、语法分析和结果展示分别封装为相对独立的模块。这样做的优点是各模块职责清晰，便于分别开发、测试和调试：词法分析器可以独立验证 token 识别是否正确；语法分析器可以在 token 流基础上独立验证 AST 构造是否正确；Web 展示模块则主要负责用户交互和结果可视化，而不直接承担词法或语法规则的实现。

本项目的总体处理流程如图 1 所示。

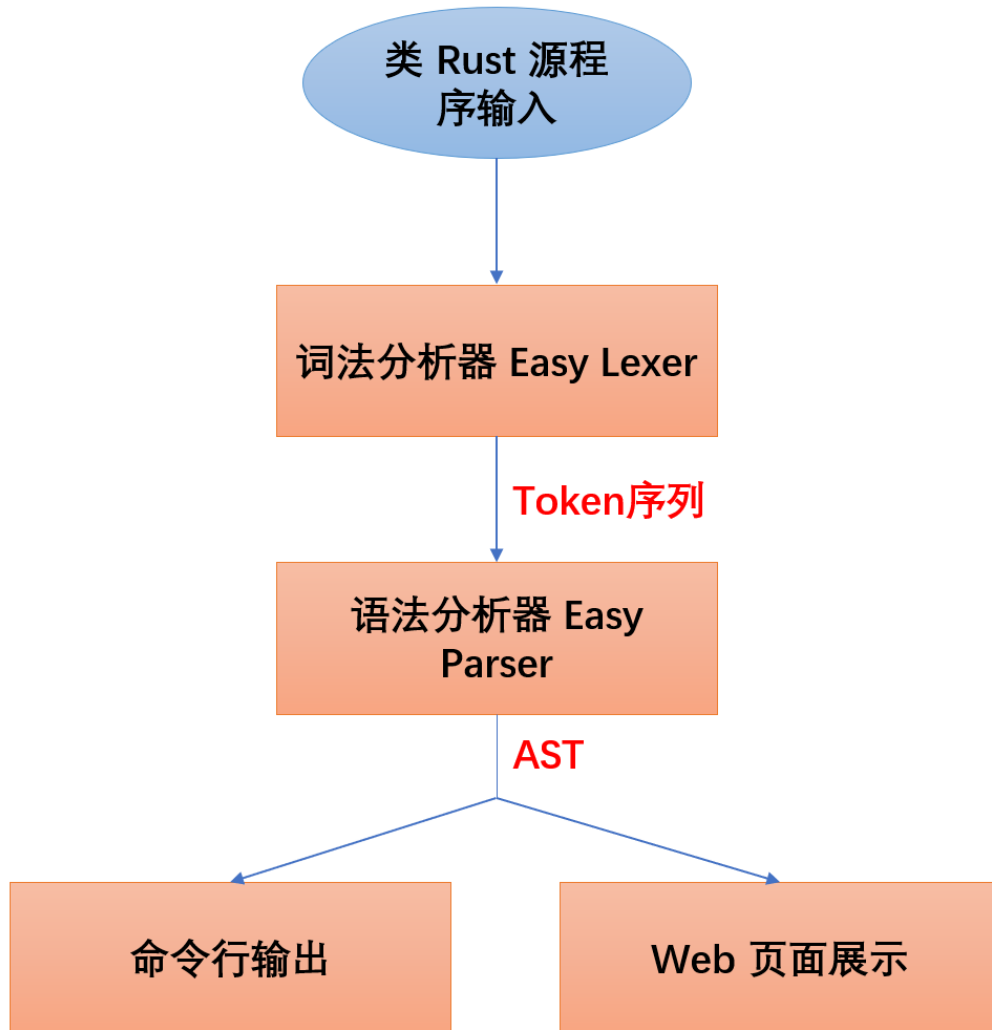


图 1: 系统总体处理流程

## 2.2 系统模块划分

根据功能职责，本项目主要划分为三个核心模块：词法分析模块、语法分析模块和 Web 展示模块。整体结构上，三个核心模块分别对应独立的 Rust crate，便于分别构建和测试。



表 1: 系统核心模块划分

| 模块名称        | 主要职责                                                              | 输出结果                     |
|-------------|-------------------------------------------------------------------|--------------------------|
| Easy_Lexer  | 读取类 Rust 源代码，完成关键字、标识符、整数、运算符、界符、分隔符、注释等词法单元的识别，并记录 token 的行列位置信息 | Token 序列和词法错误信息          |
| Easy_Parser | 调用词法分析器获取 token 流，使用递归下降方法进行语法分析，识别函数、语句、表达式、控制流等语法结构             | 抽象语法树 AST 和语法错误信息        |
| Easy_Server | 提供本地 Web 服务和分析接口，将词法分析、语法分析结果返回前端页面进行展示                           | Token 表、AST 树和错误信息的可视化结果 |

这种模块划分方式与编译器前端的自然阶段相对应。词法分析器只负责将字符流转换为 token，不直接判断复杂语法结构；语法分析器以 token 流为输入，负责识别程序层次结构；Web 服务则作为展示层，将底层分析能力包装为可交互界面。通过这种设计，系统既能满足命令行运行和自动化测试的需要，也能可视化演示及使用的需要，提高了系统的可用性和可维护性。

2.3 数据流设计

本项目的数据流围绕“源代码到 AST”的转换过程展开。用户输入的类 Rust 源代码首先以字符串形式进入系统，然后经过词法分析器转换为 token 序列。token 序列保留了源程序中的词法信息，包括 token 类别、原始文本、行号和列号。语法分析器在 token 序列基础上进行递归下降分析，并构造程序的抽象语法树。最后，AST 和错误信息会被序列化为适合输出或展示的形式。

系统的数据流可以分为以下几个阶段：

- 1. **源代码输入阶段：**用户通过命令行参数、标准输入或 Web 页面输入类 Rust 源程序；
- 2. **词法扫描阶段：**词法分析器逐字符扫描源代码，生成 token 序列，并记录可能出现的词法错误；

- 3. **语法解析阶段：**语法分析器读取 token 序列，根据语法规则识别函数、语句、表达式和类型等结构；
- 4. **AST 构造阶段：**语法分析器将识别到的语法结构组织为抽象语法树；
- 5. **结果输出阶段：**系统将 token、AST 和错误信息输出到命令行、JSON 文件或 Web 页面。

结合具体代码实现，其数据流关系如图 2 所示。

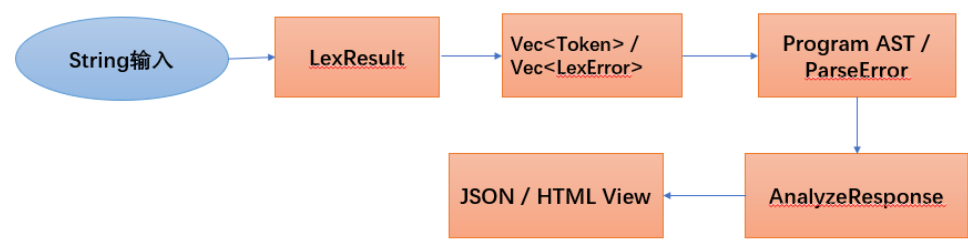


图 2: 系统数据流示意图

### 3 词法分析器设计与实现

#### 3.1 词法分析器的功能定位

词法分析是编译器前端的第一个阶段，其主要任务是将源程序的字符流转换为具有明确类别的 token 序列。对于后续语法分析器而言，词法分析器起到了屏蔽字符细节、抽象源程序结构的作用。语法分析器不再直接处理单个字符，而是基于词法分析器输出的 token 序列识别更高层次的语法结构。

本项目中的词法分析器位于 Easy\_Lexer 模块中，核心实现文件为 lib.rs。该模块面向课程要求中的类 Rust 语言语法规则，能够识别关键字、标识符、整数、赋值号、算术运算符、比较运算符、界符、分隔符、特殊符号、注释和结束符等词法单元。同时，词法分析器还会为每个 token 记录其在源程序中的行号和列号，以便在后续语法分析或错误提示中定位问题。

从输入输出关系上看，词法分析器的输入是一段 &str 类型的源代码字符串，输出是一个 LexResult 结构。该结构中包含两个部分：一是扫描得到的 token 列表，二是扫描过程中发现的词法错误列表。这样的设计使得词法分析器不仅能够返回正常分析结果，也能够出现词法错误时保留错误信息，便于前端页面或命令行程序统一展示。

### 3.2 词法分析状态转换图

词法分析器采用逐字符扫描方式。扫描器从初始状态开始，根据当前字符类别进入不同识别状态，包括标识符或关键字识别状态、整数识别状态、复合符号识别状态、注释处理状态和错误处理状态。每当识别出一个完整 token 后，扫描器将 token 加入结果序列，并重新回到初始状态继续扫描后续字符。

词法分析器的状态转换逻辑可概括为图 3 所示。

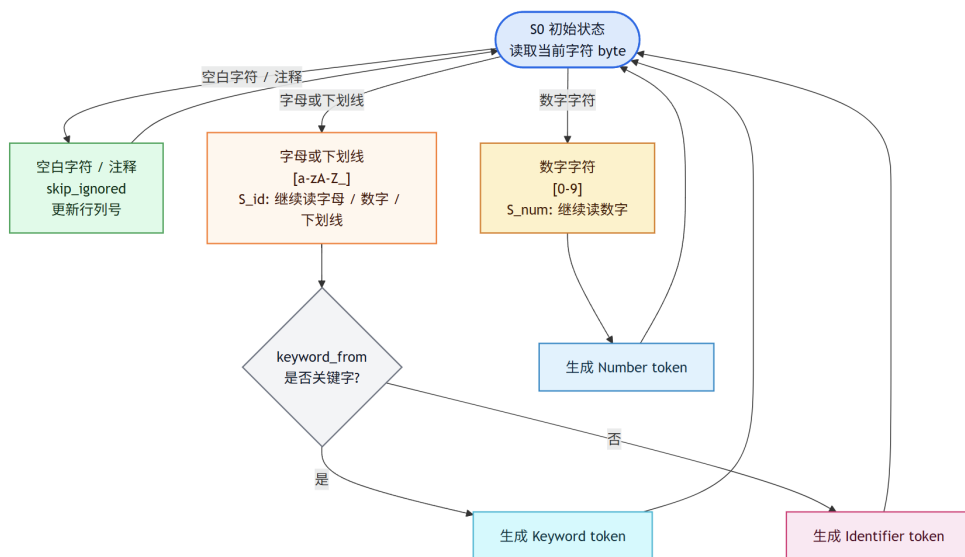


图 3: 词法分析状态转换图

而对于注释的处理，由如下状态转换图 4 所示。

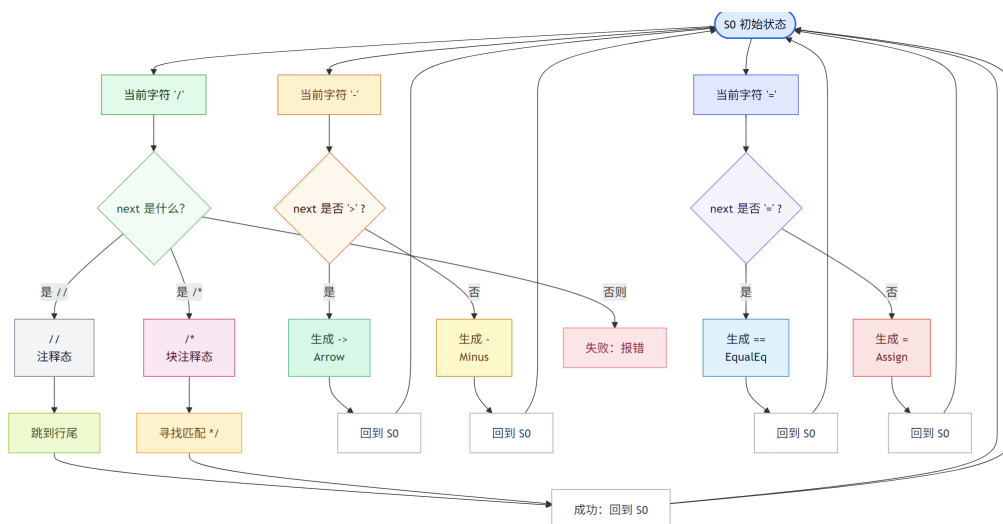


图 4: 注释处理状态转换图

### 3.3 核心数据结构设计

#### 3.3.1 位置信息 Position

为了支持错误定位和 token 位置展示，词法分析器首先定义了 Position 结构，用于记录某个 token 或词法错误在源程序中的行号和列号。

Listing 1: Position 结构定义

```
1 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
2 pub struct Position {
3     pub line: usize,
4     pub column: usize,
5 }
```

其中，line 表示当前 token 所在行，column 表示当前 token 起始位置所在列。词法分析器在扫描过程中会维护当前行列号。当读取普通字符时，列号递增；当读取换行符时，行号递增并将列号重置为 1。通过这种方式，后续一旦发现非法字符、未闭合注释或语法错误，就可以向用户报告较为准确的位置。

#### 3.3.2 Token 结构

词法分析器输出的基本单位是 token。本项目中的 token 由三部分组成：token 类别、源程序中的原始文本以及位置信息。

Listing 2: Token 结构定义

```
1 #[derive(Debug, Clone, PartialEq, Eq)]
2 pub struct Token {
3     pub kind: TokenKind,
4     pub lexeme: String,
5     pub position: Position,
6 }
```

其中，kind 表示 token 的类型，例如关键字、标识符、整数、加号、左括号等；lexeme 保存源程序中的原始字符串，例如 "if"、"main"、"123"；position 则记录该 token 在源程序中的起始位置。

这种设计同时保留了词法类别和源代码文本。词法类别供后续语法分析器判断结构使用，原始文本则便于输出 token 表、生成错误信息或进一步构造 AST 节点。

#### 3.3.3 TokenKind 枚举

TokenKind 是词法分析器中最重要的数据结构之一，用于描述所有可识别 token 的类别。本项目将 token 类型划分为关键字、标识符、数字、运算符、界符、分隔符、特殊符号和结束符等几类。

Listing 3: TokenKind 枚举定义

```
1 #[derive(Debug, Clone, PartialEq, Eq)]
2 pub enum TokenKind {
3     Keyword(Keyword),
4     Identifier,
5     Number,
6     Assign,
7     Plus,
8     .....
9 }
```

其中，关键字并没有被设计为多个独立的 `TokenKind` 变体，而是统一表示为 `Keyword(Keyword)`。这种设计将“是否为关键字”和“具体是哪一个关键字”分成两个层次：`TokenKind` 负责说明该 token 是关键字类别，`Keyword` 枚举负责说明具体关键字内容。这样可以使 token 类型结构更加清晰，也方便后续语法分析器统一处理关键字。

除关键字外，`Identifier` 表示标识符，`Number` 表示整数。运算符部分包括赋值号 `=`、算术运算符 `+` `-` `*` `/`、比较运算符 `==` `>` `>=` `<` `<=` `!=` 和引用相关符号 `&`。界符包括圆括号、花括号和方括号；分隔符包括分号、冒号和逗号；特殊符号包括函数返回箭头 `->`、点号 `.` 和区间符号 `..`；`EndMarker` 则对应课程要求中的结束符 `#`。

### 3.3.4 Keyword 枚举

课程要求中给定了一组类 Rust 语言关键字。代码中使用 `Keyword` 枚举对这些保留字进行统一管理。

Listing 4: Keyword 枚举定义

```
1 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
2 pub enum Keyword {
3     I32,
4     Let,
5     If,
6     Else,
7     While,
8     Return,
9     Mut,
10    Fn,
11    For,
12    In,
13    Loop,
14    Break,
15    Continue,
16 }
```

该枚举覆盖了课程要求中的全部关键字，包括基础类型 `i32`，变量声明关键字 `let`，控制流关键字 `if`、`else`、`while`，函数相关关键字 `fn`、`return`，可变性关键

字 `mut`，以及扩展语法中使用的 `for`、`in`、`loop`、`break`、`continue`。

在实际扫描过程中，词法分析器会先按照标识符规则读取完整单词，然后调用 `keyword_from` 函数判断该单词是否属于关键字。如果匹配成功，则生成 `TokenKind::Keyword(keyword)`，否则生成普通的 `TokenKind::Identifier`。

### 3.3.5 词法错误 `LexError` 与结果 `LexResult`

为了统一表示词法分析结果，代码定义了 `LexError` 和 `LexResult` 两个结构。

Listing 5: `LexError` 与 `LexResult` 结构定义

```
1 #[derive(Debug, Clone, PartialEq, Eq)]
2 pub struct LexError {
3     pub message: String,
4     pub position: Position,
5 }
6
7 #[derive(Debug, Default, Clone, PartialEq, Eq)]
8 pub struct LexResult {
9     pub tokens: Vec<Token>,
10    pub errors: Vec<LexError>,
11 }
```

`LexError` 中包含错误信息和错误位置。例如，当扫描过程中遇到无法识别的字符时，会生成形如 `unexpected character` 的错误；当块注释未闭合时，会生成 `unterminated block comment` 错误。

`LexResult` 同时保存 token 列表和错误列表。这种设计比直接返回 `Result<Vec<Token>, LexError>` 更适合课程项目中的展示需求。因为词法分析器在遇到某些错误后仍可以继续扫描后续字符，从而一次运行中返回更完整的 token 和错误信息。

## 3.4 词法分析器的内部状态设计

为了完成逐字符扫描，代码内部定义了 `Lexer` 结构。该结构不对外暴露，而是作为 `lex` 函数内部使用的扫描器对象。

Listing 6: `Lexer` 内部状态定义

```
1 struct Lexer<'a> {
2     input: &'a str,
3     index: usize,
4     line: usize,
5     column: usize,
6     tokens: Vec<Token>,
7     errors: Vec<LexError>,
8 }
```

其中, `input` 保存待扫描的源程序字符串, `index` 表示当前扫描到的字节位置, `line` 和 `column` 表示当前行号和列号, `tokens` 用于保存已经识别出的 token, `errors` 用于保存扫描过程中发现的词法错误。

扫描器通过 `Lexer::new(input)` 初始化。初始状态下, `index` 为 0, 表示从输入字符串开头开始扫描; `line` 和 `column` 均为 1, 表示源程序第一行第一列; token 列表和错误列表均为空。

这种内部状态设计体现了典型的手写词法分析器结构: 扫描器一边读取字符, 一边更新当前位置, 同时根据识别结果向 token 列表或错误列表中加入信息。

### 3.5 公共接口设计

词法分析器对外提供的主要接口是 `lex` 函数。

Listing 7: 词法分析器公共入口函数

```
1 pub fn lex(input: &str) -> LexResult {  
2     let mut lexer = Lexer::new(input);  
3     lexer.lex_all();  
4     LexResult {  
5         tokens: lexer.tokens,  
6         errors: lexer.errors,  
7     }  
8 }
```

该函数接收源代码字符串作为参数, 首先创建一个内部 `Lexer` 对象, 然后调用 `lex_all` 完成完整扫描, 最后将扫描器中的 token 和错误信息封装为 `LexResult` 返回。

这种接口设计较为简洁。调用方不需要关心词法分析器内部如何维护字符位置、如何读取下一个字符或如何处理注释, 只需要调用 `lex(input)` 即可获得词法分析结果。后续语法分析器、命令程序和 Web 服务都可以基于该接口复用词法分析能力。

### 3.6 主扫描流程设计

词法分析器的核心扫描逻辑位于 `lex_all` 方法中。该方法通过循环不断读取输入字符, 直到扫描到输入末尾。

在 `lex_all` 中, 扫描循环的条件是 `self.index < self.input.len()`。每轮循环开始时先调用 `skip_ignored` 跳过空白字符和注释。如果跳过后已经到达输入末尾, 则终止扫描。否则, 词法分析器记录当前 token 的起始位置 `start` 和起始下标 `start_index`, 随后通过 `peek` 查看当前字节, 并进入对应的识别分支。

### 3.7 标识符与关键字识别

标识符和关键字的识别是词法分析器中的重要部分。代码中使用两个辅助函数定义标识符规则：

Listing 8: 标识符字符判断函数

```

1 fn is_identifier_start(byte: u8) -> bool {
2     byte.is_ascii_alphabetic() || byte == b'_'
3 }
4
5 fn is_identifier_continue(byte: u8) -> bool {
6     is_identifier_start(byte) || byte.is_ascii_digit()
7 }

```

其中，标识符的首字符必须是英文字母或下划线，后续字符可以是英文字母、下划线或数字。这与课程要求中的规则“标识符由字母或下划线开头，后接字母、数字或下划线”一致。

在扫描过程中，如果当前字符满足 `is_identifier_start`，词法分析器会继续读取所有满足 `is_identifier_continue` 的后续字符，形成一个完整的字符串。随后调用 `keyword_from` 判断该字符串是否是关键字。

Listing 9: 关键字匹配函数

```

1 fn keyword_from(lexeme: &str) -> Option<Keyword> {
2     match lexeme {
3         "i32" => Some(Keyword::I32),
4         "let" => Some(Keyword::Let),
5         "if" => Some(Keyword::If),
6         "else" => Some(Keyword::Else),
7         "while" => Some(Keyword::While),
8         "return" => Some(Keyword::Return),
9         "mut" => Some(Keyword::Mut),
10        "fn" => Some(Keyword::Fn),
11        "for" => Some(Keyword::For),
12        "in" => Some(Keyword::In),
13        "loop" => Some(Keyword::Loop),
14        "break" => Some(Keyword::Break),
15        "continue" => Some(Keyword::Continue),
16        _ => None,
17    }
18 }

```

这种处理方式保证了关键字识别具有正确的边界。词法分析器不是看到 `if` 前缀就立即生成关键字，而是先读取完整单词。例如，对于输入 `if123`，完整读取结果为 `"if123"`，该字符串无法在 `keyword_from` 中匹配到关键字，因此会被识别为普通标识符。对于输入 `if=123`，扫描器首先读取到完整单词 `"if"`，由于后续的 `=` 不属于标识符字符，因此 `"if"` 会被匹配为关键字；随后 `=` 和 `123` 会分别被识别为赋值号和整数。



这一设计直接满足了课程中对 `if123` 和 `if=123` 的特殊测试要求。

### 3.8 整数识别

整数识别逻辑相对直接。当当前字符是 ASCII 数字时，词法分析器进入数字扫描分支。扫描器首先读取当前数字，然后继续读取后续所有连续数字，直到遇到非数字字符为止。最终，该连续数字串会被生成为 `TokenKind::Number` 类型的 token。

该项目中的数字目前只支持十进制整数形式，尚未支持负数字面量、小数、十六进制数或科学计数法。负号被单独识别为 `Minus`，因此表达式 `-1` 在词法层面会被拆分为减号和整数 1，其具体含义需要由语法分析阶段进一步处理。

### 3.9 复合 Token 与最长匹配策略

类 Rust 词法规则中包含若干由两个字符组成的复合符号，例如 `->`、`==`、`>=`、`<=`、`!=` 和 `..`。对于这些符号，词法分析器必须优先进行最长匹配，否则可能会将 `>=` 错误拆分为 `>` 和 `=`，或将 `..` 错误拆分为两个 `.`。

代码中使用 `match_compound_token` 方法集中处理复合 token：

Listing 10: 复合 token 匹配函数

```
1 fn match_compound_token(&self) -> Option<(TokenKind, usize)> {
2     match (self.peek(), self.peek_next()) {
3         (Some(b'-''), Some(b'>'')) => Some((TokenKind::Arrow, 2)),
4         (Some(b'='), Some(b'=')) => Some((TokenKind::EqualEqual, 2)),
5         (Some(b'>'), Some(b'=')) => Some((TokenKind::GreaterEqual, 2)),
6         (Some(b'<'), Some(b'=')) => Some((TokenKind::LessEqual, 2)),
7         (Some(b'!'), Some(b'=')) => Some((TokenKind::NotEqual, 2)),
8         (Some(b'.'), Some(b'.')) => Some((TokenKind::DotDot, 2)),
9         _ => None,
10    }
11 }
```

该函数通过同时查看当前字符和下一个字符来判断是否存在复合 token。如果匹配成功，返回对应的 `TokenKind` 和 token 宽度。当前实现中的复合 token 宽度均为 2，因此在主扫描逻辑中会连续调用两次 `advance`，将两个字符一起消费掉，再生成对应 token。

主扫描流程中，复合 token 的识别发生在单字符 token 之前。这一点非常关键。若先识别单字符 token，则 `==` 会在第一个 `=` 处被提前识别为 `Assign`，从而破坏最长匹配原则。当前实现通过先调用 `match_compound_token`，再处理单字符符号，保证了复合符号优先识别。

## 3.10 单字符 Token 识别

当当前字符既不是标识符起始字符，也不是数字，并且不能匹配复合 token 时，词法分析器会尝试将其识别为单字符 token。该部分通过 `match byte` 实现。

Listing 11: 单字符 token 识别逻辑

```
1 let kind = match byte {  
2   b'=' => Some(TokenKind::Assign),  
3   b'+' => Some(TokenKind::Plus),  
4   b'-' => Some(TokenKind::Minus),  
5   b'*' => Some(TokenKind::Star),  
6   b'/' => Some(TokenKind::Slash),  
7   b'>' => Some(TokenKind::Greater),  
8   b'<' => Some(TokenKind::Less),  
9   b'&' => Some(TokenKind::Ampersand),  
10  b'(' => Some(TokenKind::LParen),  
11  b')' => Some(TokenKind::RParen),  
12  b'{' => Some(TokenKind::LBrace),  
13  b'}' => Some(TokenKind::RBrace),  
14  b'[' => Some(TokenKind::LBracket),  
15  b']' => Some(TokenKind::RBracket),  
16  b';' => Some(TokenKind::Semicolon),  
17  b':' => Some(TokenKind::Colon),  
18  b',' => Some(TokenKind::Comma),  
19  b'.' => Some(TokenKind::Dot),  
20  b'#' => Some(TokenKind::EndMarker),  
21  _ => None,  
22 };
```

这部分覆盖了课程要求中的赋值号、算术运算符、比较运算符中的单字符形式、引用符号、各种括号、分隔符、点号和结束符。若匹配成功，则扫描器调用 `advance` 消费当前字符，并调用 `push_token` 将 token 加入列表。

## 3.11 空白字符与注释处理

源程序中的空白字符和注释不会作为有效 token 传递给语法分析器，因此词法分析器在每轮扫描开始时调用 `skip_ignored` 方法跳过这些内容。

### 3.11.1 空白字符处理

当当前字符是 ASCII 空白字符时，词法分析器直接调用 `advance` 跳过。由于 `advance` 内部会更新行号和列号，所以即使空白字符不生成 token，也不会影响后续 token 的位置记录。

### 3.11.2 单行注释处理

当词法分析器遇到 `//` 时，会进入单行注释处理逻辑。扫描器先消费两个斜杠，然后持续读取字符，直到遇到换行符或输入结束为止。单行注释中的内容不会生成 token。

该处理方式符合一般编程语言中单行注释的语义，即从 `//` 开始到当前行结束之间的内容均被忽略。

### 3.11.3 块注释处理

当词法分析器遇到 `/*` 时，会进入块注释处理逻辑。代码中不仅支持普通块注释，还通过 `depth` 变量支持嵌套块注释。扫描器在进入块注释后将 `depth` 初始化为 1；如果在注释内部再次遇到 `/*`，则将 `depth` 加 1；如果遇到 `*/`，则将 `depth` 减 1；当 `depth` 重新变为 0 时，说明最外层块注释已经闭合。

这种设计比只寻找第一个 `*/` 更健壮。例如，对于如下输入：

Listing 12: 嵌套块注释示例

```
1 /* outer /* inner */ outer */
```

词法分析器能够正确跳过整个注释，而不会在内部注释结束处提前退出。

如果扫描到输入末尾时 `depth` 仍未归零，则说明块注释未闭合。此时词法分析器会生成 `unterminated block comment` 错误，并将错误位置记录为块注释开始的位置。

## 3.12 字符读取与位置维护

词法分析器通过 `peek`、`peek_next`、`advance` 和 `position` 等辅助方法完成字符读取和位置维护。

Listing 13: 字符读取与位置维护函数

```
1 fn peek(&self) -> Option<u8> {
2     self.input.as_bytes().get(self.index).copied()
3 }
4
5 fn peek_next(&self) -> Option<u8> {
6     self.input.as_bytes().get(self.index + 1).copied()
7 }
8
9 fn advance(&mut self) -> Option<u8> {
10     let byte = self.peek()?;
11     self.index += 1;
12     if byte == b'\n' {
13         self.line += 1;
14         self.column = 1;
```

```

15     } else {
16         self.column += 1;
17     }
18     Some(byte)
19 }
20
21 fn position(&self) -> Position {
22     Position {
23         line: self.line,
24         column: self.column,
25     }
26 }

```

其中，`peek` 用于查看当前字符但不消费，`peek.next` 用于查看下一个字符，主要服务于复合 token 和注释识别。`advance` 用于真正消费当前字符，同时更新 `index`、`line` 和 `column`。`position` 则用于获取当前扫描位置，通常在识别 token 或记录错误之前调用。

需要注意的是，当前实现以字节为单位扫描，并使用 ASCII 判断函数，例如 `is_ascii_alphabetic` 和 `is_ascii_digit`。这与课程给定的类 Rust 词法规则相匹配，因为课程规则中的字母和数字均限定在 ASCII 范围内。如果后续希望支持 Unicode 标识符，则需要将当前的字节级扫描扩展为字符级扫描。

### 3.13 Token 输出与格式化

当词法分析器成功识别一个 token 后，会调用 `push_token` 方法将其加入 token 列表。

Listing 14: Token 加入结果列表

```

1 fn push_token(&mut self, kind: TokenKind, lexeme: &str, position: Position) {
2     self.tokens.push(Token {
3         kind,
4         lexeme: lexeme.to_string(),
5         position,
6     });
7 }

```

此外，代码还为 `TokenKind` 和 `Keyword` 实现了 `fmt::Display`。这使得 token 类型在命令行输出、测试结果展示或 Web 页面展示时可以转换为更直观的字符串形式。例如，`TokenKind::Identifier` 会显示为 `identifier`，`TokenKind::Keyword(Keyword::If)` 会显示为 `keyword(if)`。

这种格式化实现虽然不直接影响词法分析的正确性，但能够显著提高结果输出的可读性，有助于实验报告截图展示和调试分析。

### 3.14 错误处理机制

词法分析器的错误处理主要体现在两个方面：非法字符处理和未闭合块注释处理。

当当前字符无法匹配任何合法 token 时，扫描器会将其视为非法字符，并生成一个 `LexError`：

Listing 15: 非法字符错误处理

```
1 let ch = byte as char;
2 self.errors.push(LexError {
3     message: format!("unexpected character '{ch}'"),
4     position: start,
5 });
6 self.advance();
```

这里的设计是“记录错误并继续扫描”。也就是说，词法分析器不会因为遇到一个非法字符就立即终止，而是将错误加入 `errors` 列表，然后跳过该字符继续处理后续输入。这样可以在一次扫描中尽可能发现更多词法问题。

另一类错误是未闭合块注释。当扫描器进入块注释后，如果直到输入结束都没有找到匹配的结束符 `*/`，则会生成 `unterminated block comment` 错误。该错误的位置被设置为块注释开始处，从而方便用户定位问题。

### 3.15 词法分析器实现特点

综合来看，本项目词法分析器具有以下几个实现特点。

第一，数据结构清晰。代码将位置信息、token、token 类型、关键字、错误和最终结果分别定义为独立结构，使词法分析器输出具有较好的可读性和可扩展性。

第二，扫描逻辑直接。词法分析器采用手写逐字符扫描方式，通过 `peek`、`peek_next` 和 `advance` 控制扫描过程。这种方式实现简单、流程明确，适合课程项目中展示词法分析的基本原理。

第三，关键字识别方式正确。词法分析器先读取完整标识符候选串，再判断其是否为关键字，因此能够正确处理 `if123` 这类关键字前缀标识符问题。

第四，支持最长匹配。复合 token 的识别被放在单字符 token 之前，保证 `>=`、`==`、`->`、`..` 等符号不会被错误拆分。

第五，注释处理较完整。词法分析器能够跳过单行注释和块注释，并且块注释部分通过深度计数支持嵌套结构。对于未闭合块注释，也能够给出明确错误信息。

第六，错误处理较友好。词法分析器在遇到非法字符时记录错误并继续扫描，使调用方可以在一次运行中获得更完整的分析结果。

## 4 语法分析器设计与实现

### 4.1 语法分析器的功能定位

语法分析是编译器前端中承接词法分析的重要阶段。词法分析器将源程序字符流转换为 token 序列，而语法分析器则在 token 序列基础上，根据语言文法判断程序结构是否合法，并构造抽象语法树。相比词法分析关注“每个词是什么”，语法分析更关注“这些词如何组成合法的程序”。

本项目中的语法分析器位于 `Easy.Parser` 模块中，其输入为 `Easy.Lexer` 输出的 token 序列，输出为表示程序结构的抽象语法树。语法分析器能够识别类 Rust 语言中的函数声明、参数列表、返回类型、语句块、变量声明、赋值语句、返回语句、表达式、函数调用、if 选择结构、while 循环等课程强制要求内容。同时，代码还支持 for 循环、loop 循环、break、continue、引用、可变引用、解引用、数组、元组、表达式块、if 表达式和 loop 表达式等扩展结构。

从实现方法上看，本项目采用手写递归下降分析器。递归下降方法的基本思想是：为文法中的主要非终结符设计对应的解析函数，通过函数调用关系体现语法结构。例如，程序由函数声明序列组成，因此 `parse_program` 会反复调用 `parse_function_decl`；函数体由语句块组成，因此函数声明解析过程中会继续调用 `parse_block`；表达式又按照不同优先级拆分为加减表达式、乘除表达式、一元表达式和基本表达式等层次。这样的设计结构清晰，便于将课程给定的产生式规则映射到代码实现中。

总体来看，本语法分析器的函数调用状态转移图如下所示：

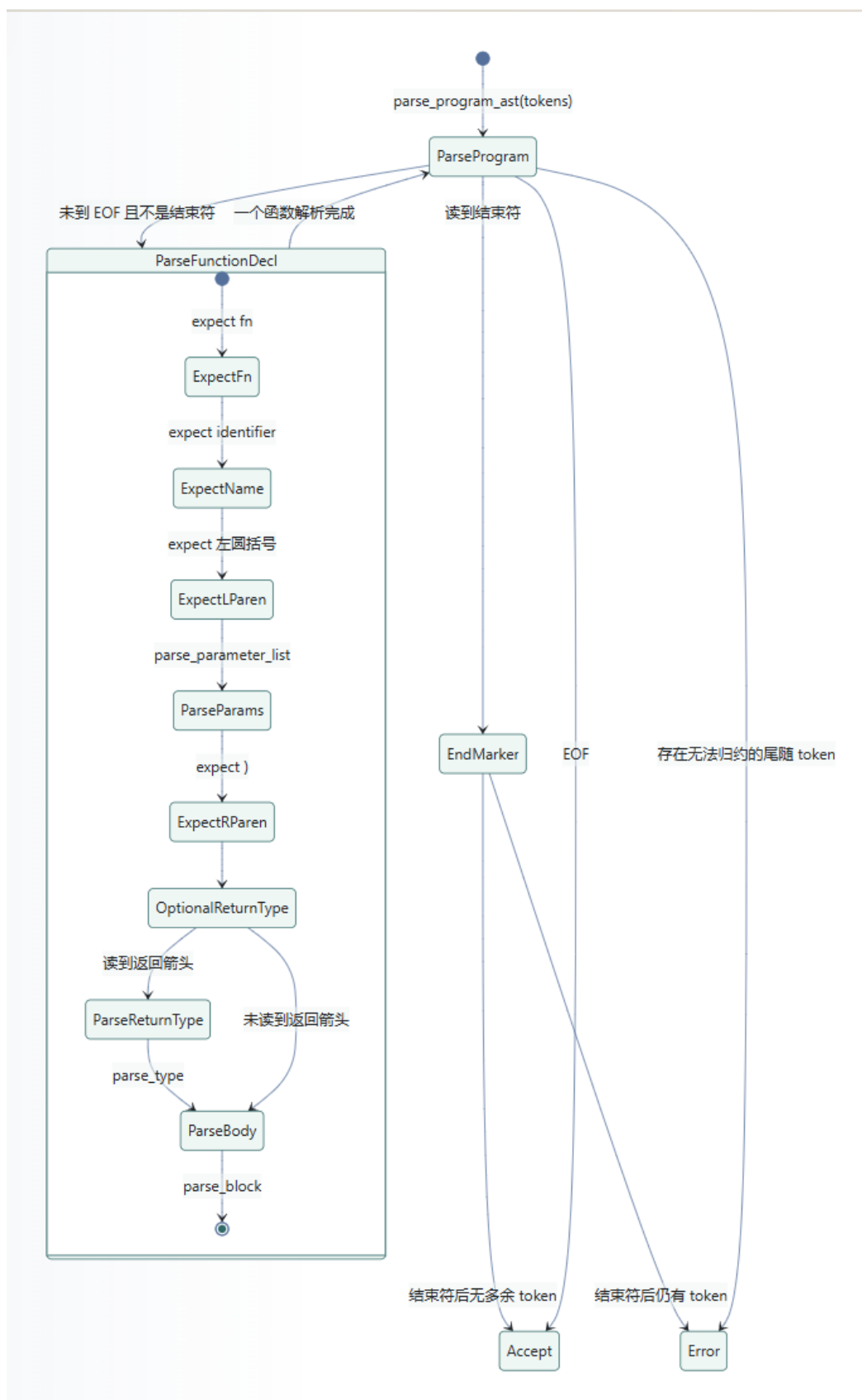


图 5: 语法分析状态转换图

## 4.2 语法分析器公共接口设计

语法分析器对外提供了两个主要接口：`parse_tokens` 和 `parse_program_ast`。

Listing 16: 语法分析器公共接口

```
1 #[allow(dead_code)]
2 pub fn parse_tokens(tokens: &[Token]) -> Result<(), ParseError> {
3     Parser::new(tokens).parse_program().map(|_| ())
4 }
5
6 pub fn parse_program_ast(tokens: &[Token]) -> Result<Program, ParseError> {
7     Parser::new(tokens).parse_program()
8 }
```

其中，`parse_tokens` 主要用于判断 token 序列是否能够被成功解析。如果解析成功，则返回 `Ok(())`；如果出现语法错误，则返回 `ParseError`。该接口适合用于测试语法合法性。

`parse_program_ast` 则返回完整的 Program AST 节点，是本项目中生成语法树的核心接口。命令行工具或 Web 服务可以调用该函数，将词法分析器生成的 token 序列转换为可序列化、可展示的 AST 结构。

这种接口设计将“语法检查”和“AST 生成”进行了区分。前者适合快速判断语法是否正确，后者适合用于后续展示、序列化和进一步分析。

## 4.3 语法错误表示

语法分析过程中可能出现缺少括号、缺少分号、表达式不完整、函数声明格式错误等问题。为了统一描述这些错误，代码定义了 `ParseError` 结构。

Listing 17: `ParseError` 结构定义

```
1 #[derive(Debug, Clone, PartialEq, Eq)]
2 pub struct ParseError {
3     pub message: String,
4     pub position: Position,
5 }
```

其中，`message` 表示错误信息，`position` 表示错误所在位置。该位置直接复用词法分析器中 token 携带的 `Position`，因此语法错误可以准确定位到源代码中的行号和列号。

代码还为 `ParseError` 实现了 `fmt::Display`：

Listing 18: `ParseError` 的显示格式

```
1 impl fmt::Display for ParseError {
2     fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
3         write!(
4             f,
```



```
5         "syntax error at {}:{}: {}",  
6         self.position.line, self.position.column, self.message  
7     )  
8 }  
9 }
```

该实现使得错误信息可以以类似 `syntax error at line:column: message` 的形式输出，便于命令行和 Web 页面展示。相比只提示“语法错误”，这种带位置信息的错误更有助于用户定位源程序中的问题。

## 4.4 AST 总体结构设计

抽象语法树是语法分析器的核心输出。相比具体语法树，AST 会省略一部分语法细节，例如括号、逗号、分号等纯结构性符号，而保留程序真正的层次结构和语义相关信息。本项目中的 AST 节点均支持 `Serialize`，因此可以方便地转换为 JSON，在命令行或 Web 页面中展示。

从整体上看，本项目的 AST 主要包括以下几类节点：

- `Program`：表示整个程序；
- `FunctionDecl`：表示函数声明；
- `Binding`：表示变量绑定或参数绑定；
- `Block`：表示语句块；
- `Statement`：表示语句；
- `Expr`：表示表达式；
- `TypeNode`：表示类型；
- `UnaryOp` 和 `BinaryOp`：表示一元和二元运算符；
- `ElseBranch`：表示 if 表达式中的 else 分支。

这些节点共同描述了类 Rust 程序的主要结构。顶层 `Program` 包含若干函数声明；每个函数声明包含函数名、参数列表、返回类型和函数体；函数体是一个 `Block`，其中包含若干语句和可选的尾表达式；语句和表达式则进一步描述变量声明、赋值、控制流和运算结构。

## 4.5 Program 与 FunctionDecl 设计

程序顶层结构由 Program 表示：

Listing 19: Program 节点定义

```
1 #[derive(Debug, Clone, Serialize)]
2 pub struct Program {
3     pub kind: &'static str,
4     pub functions: Vec<FunctionDecl>,
5 }
```

其中，kind 字段用于在 JSON 输出中标识节点类型，functions 保存程序中的函数声明列表。类 Rust 源程序的顶层结构主要由函数声明组成，因此 Program 节点的设计较为直接。

函数声明由 FunctionDecl 表示：

Listing 20: FunctionDecl 节点定义

```
1 #[derive(Debug, Clone, Serialize)]
2 pub struct FunctionDecl {
3     pub kind: &'static str,
4     pub name: String,
5     pub params: Vec<Binding>,
6     #[serde(rename = "returnType")]
7     pub return_type: Option<TypeNode>,
8     pub body: Block,
9 }
```

其中，name 表示函数名，params 表示参数列表，return\_type 表示可选返回类型，body 表示函数体。返回类型被设计为 Option<TypeNode>，是因为类 Rust 函数既可以显式声明返回类型，例如 fn f() -> i32，也可以没有返回类型，例如 fn f()。

在 JSON 序列化时，代码通过 #[serde(rename = "returnType")] 将 return\_type 字段重命名为 returnType，使输出结果更适合前端展示。

## 4.6 Binding 与类型节点设计

### 4.6.1 Binding 节点

Binding 用于统一表示函数参数和变量声明中的绑定信息。

Listing 21: Binding 节点定义

```
1 #[derive(Debug, Clone, Serialize)]
2 pub struct Binding {
3     pub kind: &'static str,
4     pub mutable: bool,
5     pub name: String,
```

```

6   pub ty: Option<TypeNode>,
7 }

```

其中，`mutable` 表示是否带有 `mut` 修饰，`name` 表示变量名或参数名，`ty` 表示可选类型标注。将参数和变量声明都抽象为 `Binding` 有助于复用解析逻辑。例如，函数参数和 `let` 变量声明都可以通过 `parse_binding` 解析，只是在是否必须带类型上有所不同。

### 4.6.2 TypeNode 节点

类型由 `TypeNode` 枚举表示：

Listing 22: `TypeNode` 节点定义

```

1  #[derive(Debug, Clone, Serialize)]
2  #[serde(tag = "kind")]
3  pub enum TypeNode {
4      Named {
5          name: String,
6      },
7      Reference {
8          mutable: bool,
9          ty: Box<TypeNode>,
10     },
11     Array {
12         element: Box<TypeNode>,
13         length: usize,
14     },
15     Tuple {
16         elements: Vec<TypeNode>,
17     },
18     Unit,
19 }

```

当前实现支持以下类型结构：

- **Named:** 命名类型，例如 `i32`；
- **Reference:** 引用类型，例如 `&i32` 或 `&mut i32`；
- **Array:** 数组类型，例如 `[i32;3]`；
- **Tuple:** 元组类型，例如 `(i32,i32)`；
- **Unit:** 空元组类型 `()`。

虽然课程最低要求中只要求支持 `i32`，但当前实现已经扩展到引用、数组、元组和 `unit` 类型。需要注意的是，这里的类型节点主要用于语法结构表达，并不代表项目已经实现完整类型检查。变量类型是否匹配、数组元素类型是否一致、函数返回类型是否正确等问题仍属于后续语义分析范围。

## 4.7 语句节点设计

语句由 `Statement` 枚举表示。代码中使用 `#[serde(tag = "kind")]`，使不同语句类型在 JSON 输出中带有明确的 `kind` 字段。

Listing 23: Statement 节点定义

```
1  #[derive(Debug, Clone, Serialize)]
2  #[serde(tag = "kind")]
3  pub enum Statement {
4      Empty,
5      Let {
6          binding: Binding,
7          init: Option<Box<Expr>>,
8      },
9      Assign {
10         target: Box<Expr>,
11         value: Box<Expr>,
12     },
13     Expr {
14         expr: Box<Expr>,
15     },
16     Return {
17         value: Option<Box<Expr>>,
18     },
19     While {
20         condition: Box<Expr>,
21         body: Block,
22     },
23     For {
24         binding: Binding,
25         iterable: Box<Expr>,
26         body: Block,
27     },
28     Break {
29         value: Option<Box<Expr>>,
30     },
31     Continue,
32 }
```

该结构覆盖了课程要求中的空语句、变量声明语句、赋值语句、表达式语句、返回语句和 `while` 循环，也支持扩展语法中的 `for` 循环、`break` 和 `continue`。

其中，`Let` 语句中的 `init` 被设计为可选值，因此既能表示 `let mut a:i32;`，也能表示 `let mut a:i32=1;`。`Return` 语句中的 `value` 也是可选值，因此既能表示 `return;`，也能表示 `return expr;`。`Break` 同样支持可选表达式，从而兼容 `loop` 表达式中的 `break value;` 形式。

## 4.8 表达式节点设计

表达式由 `Expr` 枚举表示（篇幅原因，下面的函数只列出了一部分内容，省略了一些枚举成员）：

Listing 24: Expr 节点定义

```

1  #[derive(Debug, Clone, Serialize)]
2  #[serde(tag = "kind")]
3  pub enum Expr {
4      Identifier {
5          name: String,
6      },
7      Number {
8          value: String,
9      },
10     Unary {
11         op: UnaryOp,
12         expr: Box<Expr>,
13     }
14     .....
15 }
```

该表达式设计覆盖范围较广。其中，`Identifier` 表示变量名，`Number` 表示数字字面量，`Unary` 表示一元表达式，`Binary` 表示二元表达式，`Call` 表示函数调用，`Index` 表示数组索引，`Field` 表示元组字段访问，`Array` 和 `Tuple` 分别表示数组和元组字面量，`Block`、`If` 和 `Loop` 则体现了 Rust 中“块、if、loop 可以作为表达式”的特点。

此外，`Expr` 还定义了两个辅助方法：

Listing 25: 表达式辅助判断函数

```

1  impl Expr {
2      fn is_assignable(&self) -> bool {
3          match self {
4              Self::Identifier { .. } => true,
5              Self::Unary {
6                  op: UnaryOp::Deref, ..
7              } => true,
8              Self::Index { .. } => true,
9              Self::Field { .. } => true,
10             _ => false,
11         }
12     }
13
14     fn can_stand_without_semicolon(&self) -> bool {
15         matches!(
16             self,
17             Self::Block { .. } | Self::If { .. } | Self::Loop { .. }
18         )
19     }
20 }
```

`is_assignable` 用于判断一个表达式是否可以出现在赋值语句左侧。当前实现允许标识符、解引用表达式、数组索引和字段访问作为赋值目标，而不允许数字、二元表达式、函数调用等表达式作为赋值目标。因此，类似 `1=2;` 的输入会被识别为语法错误。

`can_stand_without_semicolon` 用于处理块表达式、`if` 表达式和 `loop` 表达式。在 Rust 风格语法中，某些表达式结构可以在语句块中不带分号地出现，并作为块的尾表达式或独立表达式处理。该函数为块解析提供了判断依据。

## 4.9 Parser 内部状态设计

语法分析器内部使用 `Parser` 结构维护 `token` 序列和当前解析位置。

Listing 26: Parser 内部状态定义

```
1 struct Parser<'a> {
2     tokens: &'a [Token],
3     current: usize,
4 }
```

其中，`tokens` 是词法分析器输出的 `token` 切片，`current` 表示当前正在查看的 `token` 下标。语法分析器通过 `peek_kind` 查看当前 `token` 类型，通过 `advance` 消费当前 `token`，通过 `match_simple`、`expect_simple`、`match_keyword`、`expect_keyword` 等方法实现匹配和错误报告。

这种设计是递归下降分析器的典型实现方式。解析函数之间共享同一个 `current` 指针，每当成功识别一个 `token`，就推进指针；如果当前 `token` 不符合预期，则构造 `ParseError` 返回。

## 4.10 程序与函数声明解析

### 4.10.1 程序解析

程序解析从 `parse_program` 开始。

Listing 27: 程序解析函数

```
1 fn parse_program(mut self) -> Result<Program, ParseError> {
2     let mut functions = Vec::new();
3
4     while !self.is_at_end() && !self.check_simple(SimpleTokenKind::EndMarker) {
5         functions.push(self.parse_function_decl()?);
6     }
7
8     if self.match_simple(SimpleTokenKind::EndMarker) && !self.is_at_end() {
9         return Err(self.error_here("unexpected tokens after '#'"));
10    }
11 }
```

```

12     if !self.is_at_end() {
13         return Err(self.error_here("unexpected trailing tokens"));
14     }
15
16     Ok(Program::new(functions))
17 }

```

该函数反复解析函数声明，直到 token 流结束或遇到结束符 #。如果遇到结束符后仍然存在其他 token，则报告 `unexpected tokens after #` 错误。该设计支持课程要求中的结束符，同时保证结束符之后不允许再出现额外程序内容。

#### 4.10.2 函数声明解析

函数声明由 `parse_function_decl` 解析。

Listing 28: 函数声明解析函数

```

1 fn parse_function_decl(&mut self) -> Result<FunctionDecl, ParseError> {
2     self.expect_keyword(Keyword::Fn)?;
3     let name = self.expect_identifier("expected function name after 'fn'");
4     self.expect_simple(SimpleTokenKind::LParen, "expected '(' after function
5         name");
6     let params = self.parse_parameter_list()?;
7     self.expect_simple(
8         SimpleTokenKind::RParen,
9         "expected ')' after function parameter list",
10    );
11
12    let return_type = if self.match_simple(SimpleTokenKind::Arrow) {
13        Some(self.parse_type())
14    } else {
15        None
16    };
17
18    let body = self.parse_block()?;
19    Ok(FunctionDecl::new(name, params, return_type, body))
20 }

```

函数声明解析严格对应类 Rust 函数结构：首先匹配 `fn` 关键字，然后读取函数名，解析圆括号中的参数列表，再解析可选的返回类型，最后解析函数体语句块。例如：

Listing 29: 函数声明示例

```

1 fn base(mut a:i32) -> i32 {
2     return a;
3 }

```

该输入会被解析为一个 `FunctionDecl` 节点，其中函数名为 `base`，参数列表中包含一个可变参数 `a:i32`，返回类型为 `i32`，函数体为一个 `Block`。

## 4.11 参数与变量绑定解析

参数列表由 `parse_parameter_list` 解析。若当前 token 为右括号，则说明参数列表为空；否则循环解析一个或多个参数绑定，并使用逗号分隔。

Listing 30: 参数列表解析函数

```

1 fn parse_parameter_list(&mut self) -> Result<Vec<Binding>, ParseError> {
2     let mut params = Vec::new();
3
4     if self.check_simple(SimpleTokenKind::RParen) {
5         return Ok(params);
6     }
7
8     loop {
9         let binding = self.parse_binding(true)?;
10        if binding.ty.is_none() {
11            return Err(self.error_here("expected ':' after parameter name"));
12        }
13        params.push(binding);
14
15        if !self.match_simple(SimpleTokenKind::Comma) {
16            break;
17        }
18    }
19
20    Ok(params)
21 }

```

参数绑定和变量绑定共用 `parse_binding` 函数。

Listing 31: 绑定解析函数

```

1 fn parse_binding(&mut self, require_type: bool) -> Result<Binding, ParseError> {
2     let mutable = self.match_keyword(Keyword::Mut);
3     let name = self.expect_identifier("expected variable name");
4     let ty = if self.match_simple(SimpleTokenKind::Colon) {
5         Some(self.parse_type())
6     } else {
7         None
8     };
9
10    if require_type && ty.is_none() {
11        return Err(self.error_here("expected ':' after parameter name"));
12    }
13
14    Ok(Binding::new(name, mutable, ty))
15 }

```

该函数首先尝试匹配可选的 `mut`，然后读取变量名，最后解析可选类型标注。如果 `require_type` 为 `true`，则要求必须带类型。这正好符合函数参数和变量声明之间的差异：函数参数必须写出类型，而 `let` 变量声明可以在扩展语法中省略类型。



## 4.12 类型解析

类型解析由 `parse_type` 完成。该函数支持命名类型、引用类型、数组类型、元组类型和 `unit` 类型。

Listing 32: 类型解析函数结构示意图

```

1 fn parse_type(&mut self) -> Result<TypeNode, ParseError> {
2     if self.match_keyword(Keyword::I32) {
3         return Ok(TypeNode::Named {
4             name: "i32".to_string(),
5         });
6     }
7
8     if self.match_simple(SimpleTokenKind::Ampersand) {
9         let mutable = self.match_keyword(Keyword::Mut);
10        return Ok(TypeNode::Reference {
11            mutable,
12            ty: Box::new(self.parse_type()),
13        });
14    }
15
16    if self.match_simple(SimpleTokenKind::LBracket) {
17        // parse array type: [Type; NUM]
18    }
19
20    if self.match_simple(SimpleTokenKind::LParen) {
21        // parse tuple type or unit type
22    }
23
24    Err(self.error_here("expected a type"))
25 }

```

对于 `i32`，解析器生成 `TypeNode::Named`。对于引用类型，解析器先匹配 `&`，再判断是否带有 `mut`，然后递归调用 `parse_type` 解析被引用的类型。例如，`&mut i32` 会被解析为一个可变引用类型，其内部类型为 `i32`。

数组类型的形式为：

Listing 33: 数组类型示例

```
1 [i32;3]
```

解析器会在左方括号后解析元素类型，然后要求出现分号，再读取数组长度数字，最后匹配右方括号。

元组类型的形式为：

Listing 34: 元组类型示例

```
1 (i32, i32)
```

解析器通过逗号判断其为元组类型。如果圆括号内部为空，即 `()`，则解析为

`TreeNode::Unit`。如果元组类型中只有第一个类型而没有逗号，代码会报错，因为元组类型要求第一个元素类型后必须有逗号。

### 4.13 语句块与尾表达式解析

Rust 风格语言中，语句块不仅可以包含若干语句，还可以包含一个不带分号的尾表达式。本项目中的 `Block` 结构正是为此设计的：

Listing 35: Block 节点定义

```
1 #[derive(Debug, Clone, Serialize)]
2 pub struct Block {
3     pub kind: &'static str,
4     pub statements: Vec<Statement>,
5     pub tail: Option<Box<Expr>>,
6 }
```

其中，`statements` 保存普通语句，`tail` 保存可选尾表达式。例如：

Listing 36: 带尾表达式的语句块

```
1 {
2     let t = 1;
3     t
4 }
```

该块中 `let t = 1;` 是普通语句，而最后的 `t` 没有分号，因此会被解析为尾表达式。

语句块解析由 `parse_block` 完成。其基本逻辑是：先匹配左花括号，然后不断解析块内部内容，直到遇到右花括号。

这里的设计体现了类 Rust 语法的特点：块内部内容既可能以关键字开头，例如 `let`、`return`、`while`，也可能以表达式开头，例如变量、数字、块表达式或 `if` 表达式。因此，解析器首先使用 `starts_forced_statement` 判断当前 `token` 是否必然属于语句开头。如果是，则调用 `parse_statement`；否则先尝试解析表达式，再根据后续 `token` 判断它是赋值语句、表达式语句还是尾表达式。

### 4.14 语句解析

普通语句由 `parse_statement` 解析。该函数通过检查当前 `token` 的类型分派到不同语句分支。按照空语句、变量声明语句、返回语句、`while` 循环、`for` 循环、`break` 和 `continue` 的顺序进行检查，最后如果都不匹配，则报告 `expected a statement` 错误。

#### 4.14.1 空语句

当当前 token 为分号时，解析器生成 `Statement::Empty`。这对应课程规则中的空语句。

#### 4.14.2 变量声明语句

当当前 token 为 `let` 时，解析器调用 `parse_binding(false)` 解析变量绑定，然后判断是否存在初始化表达式。如果存在赋值号，则继续解析初始化表达式；最后要求以分号结束。

该设计同时支持以下形式：

Listing 37: 变量声明语句示例

```
1 let mut a:i32;  
2 let c=1;  
3 let mut b:i32=2;
```

#### 4.14.3 返回语句

当当前 token 为 `return` 时，解析器判断下一个 token 是否为分号。如果是，则解析为无返回值的 `return;`；否则解析一个返回表达式，并要求最后出现分号。因此当前实现同时支持：

Listing 38: 返回语句示例

```
1 return;  
2 return a+1;
```

#### 4.14.4 while 循环

当当前 token 为 `while` 时，解析器先解析条件表达式，再解析循环体语句块，最终构造 `Statement::While` 节点。

Listing 39: while 循环示例

```
1 while a != 0 {  
2     a = a - 1;  
3 }
```

#### 4.14.5 for 循环

扩展语法中还支持 `for` 循环。解析器在匹配 `for` 后，先解析循环变量绑定，再要求出现 `in` 关键字，然后解析可选代表表达式和循环体。

Listing 40: for 循环示例

```

1 for mut i in 0..a {
2     if i==2 { continue; }
3 }

```

可迭代结构由 `parse_iterable` 解析。若表达式后出现 `..`，则构造 `BinaryOp::Range` 表达式，用于表示区间。

#### 4.14.6 break 与 continue

`break` 支持可选表达式，因此既可以表示普通循环跳出，也可以用于 `loop` 表达式返回值：

Listing 41: break 和 continue 示例

```

1 break;
2 break v;
3 continue;

```

`continue` 后不允许带表达式，必须直接以分号结束。

### 4.15 赋值语句解析与左值检查

赋值语句并没有在 `parse_statement` 中直接作为关键字分支处理，而是在 `parse_block` 中通过“先解析表达式，再判断是否跟随赋值号”的方式处理。这是因为赋值语句的开头可能是多种表达式形式，例如：

Listing 42: 赋值左侧示例

```

1 a = 1;
2 *a = 2;
3 arr[0] = 3;
4 pair.0 = 4;

```

解析器先解析出左侧表达式，如果后面出现赋值号，则判断该表达式是否可赋值：

Listing 43: 赋值左侧检查逻辑

```

1 if self.match_simple(SimpleTokenKind::Assign) {
2     if !expr.is_assignable() {
3         return Err(self.error_here("left side of assignment is not assignable"))
4     }
5     let value = self.parse_expression()?;
6     self.expect_simple(
7         SimpleTokenKind::Semicolon,
8         "expected ';' after assignment statement",
9     )?;
10    statements.push(Statement::Assign {

```

```

11     target: Box::new(expr),
12     value: Box::new(value),
13 });
14     continue;
15 }

```

该设计能够避免非法赋值目标。例如：

Listing 44: 非法赋值示例

```

1 1 = 2;

```

由于数字表达式不满足 `is_assignable`，解析器会报告 `left side of assignment is not assignable` 错误。这虽然还不是完整语义分析，但已经在语法结构层面对赋值左侧进行了基本限制。

## 4.16 表达式解析与优先级处理

表达式解析是语法分析器中最复杂的部分之一。为了正确处理运算符优先级，本项目将表达式解析拆分为多个层次：

Listing 45: 表达式解析层次

```

1 parse_expression()
2     -> parse_additive()
3         -> parse_multiplicative()
4             -> parse_unary()
5                 -> parse_postfix()
6                     -> parse_primary()

```

当前实现中，比较运算位于 `parse_expression` 层，加减运算位于 `parse_additive` 层，乘除运算位于 `parse_multiplicative` 层，一元运算位于 `parse_unary` 层，函数调用、索引和字段访问等后缀运算位于 `parse_postfix` 层，数字、标识符、括号表达式、数组、元组、块、if 和 loop 等基本结构位于 `parse_primary` 层。

这种分层设计保证了表达式优先级的正确性。例如，对于表达式：

Listing 46: 表达式优先级示例

```

1 a + 1 * 2

```

解析器会先在乘除层将 `1*2` 组合为一个乘法表达式，再在加减层与 `a` 组合为加法表达式。因此最终 AST 能够正确体现“乘法优先于加法”。

### 4.16.1 比较表达式

比较运算由 `parse_expression` 中的循环处理。该函数先解析一个加减表达式，然后不断匹配比较运算符：

Listing 47: 比较表达式解析

```

1 fn parse_expression(&mut self) -> Result<Expr, ParseError> {
2     let mut expr = self.parse_additive()?;
3
4     while let Some(op) = self.match_comparison_op() {
5         let right = self.parse_additive()?;
6         expr = Expr::Binary {
7             left: Box::new(expr),
8             op,
9             right: Box::new(right),
10        };
11    }
12
13    Ok(expr)
14 }

```

比较运算符包括 <、<=、>、>=、== 和 !=。这些运算符统一被转换为 BinaryOp 中的对应枚举值。

#### 4.16.2 加减表达式

加减表达式由 parse\_additive 解析。该函数先解析乘除表达式，再循环处理 + 和 -。

这种写法使加减运算天然具有左结合性。例如，a-b-c 会被解析为 (a-b)-c。

#### 4.16.3 乘除表达式

乘除表达式由 parse\_multiplicative 解析。该函数与加减表达式类似，但处理的是 \* 和 /。由于 parse\_additive 会调用 parse\_multiplicative，因此乘除运算的优先级高于加减运算。

#### 4.16.4 一元表达式

一元表达式由 parse\_unary 解析，支持引用、可变引用、解引用和取负操作：

Listing 48: 一元表达式示例

```

1 &a
2 &mut a
3 *a
4 -a

```

其中，& 会生成 UnaryOp::Ref，&mut 会生成 UnaryOp::RefMut，\* 会生成 UnaryOp::Deref，- 会生成 UnaryOp::Neg。引用和解引用表达式递归调用 parse\_unary，从而支持连续一元运算。

## 4.17 后缀表达式解析

后缀表达式由 `parse_postfix` 处理。该函数先解析一个基本表达式，然后循环判断其后是否跟随函数调用、数组索引或字段访问。

Listing 49: 后缀表达式解析函数结构示意图

```

1 fn parse_postfix(&mut self) -> Result<Expr, ParseError> {
2     let mut expr = self.parse_primary()?;
3
4     loop {
5         if self.check_simple(SimpleTokenKind::LParen) && Self::is_callable(&expr) {
6             // parse function call
7             continue;
8         }
9
10        if self.match_simple(SimpleTokenKind::LBracket) {
11            // parse array index
12            continue;
13        }
14
15        if self.match_simple(SimpleTokenKind::Dot) {
16            // parse tuple field
17            continue;
18        }
19
20        break;
21    }
22
23    Ok(expr)
24 }
```

该设计使后缀操作可以连续出现。例如，理论上可以解析类似 `arr[0].1` 这样的链式结构。当前实现中，函数调用要求 callee 必须是标识符、字段访问或索引表达式，这由 `is_callable` 函数控制。

数组索引形如：

Listing 50: 数组索引示例

```
1 arr[0]
```

元组字段访问形如：

Listing 51: 元组字段访问示例

```
1 pair.0
```

字段访问中，点号后必须是数字文本，这与 Rust 元组字段访问的形式一致。

## 4.18 基本表达式解析

基本表达式由 `parse_primary` 解析，是表达式解析的底层入口。该函数支持数字、标识符、块表达式、if 表达式、loop 表达式、数组字面量、元组字面量和括号表达式。

### 4.18.1 数字与标识符

当当前 token 为 `Number` 时，解析器生成 `Expr::Number`；当当前 token 为 `Identifier` 时，生成 `Expr::Identifier`。

### 4.18.2 块表达式

如果当前 token 为左花括号，解析器直接调用 `parse_block`，并将结果包装为 `Expr::Block`。这体现了 Rust 中块可以作为表达式的特点。

### 4.18.3 if 表达式

当当前 token 为 `if` 时，解析器调用 `parse_if_after_keyword` 解析 if 表达式。

Listing 52: if 表达式解析函数

```

1 fn parse_if_after_keyword(&mut self) -> Result<Expr, ParseError> {
2     let condition = self.parse_expression()?;
3     let then_branch = self.parse_block()?;
4     let else_branch = if self.match_keyword(Keyword::Else) {
5         if self.match_keyword(Keyword::If) {
6             ElseBranch::ElseIf {
7                 expr: Box::new(self.parse_if_after_keyword()),
8             }
9         } else {
10            ElseBranch::Block {
11                block: self.parse_block()?,
12            }
13        }
14    } else {
15        ElseBranch::None
16    };
17
18    Ok(Expr::If {
19        condition: Box::new(condition),
20        then_branch,
21        else_branch,
22    })
23 }
```

该函数支持无 `else` 的 `if`、带 `else` 的 `if`，以及链式 `else if`。需要注意的是，当前实现将 `if` 统一作为表达式节点处理。当 `if` 出现在语句块中并带有分号或作为独立结构出现时，会在块解析逻辑中进一步转换为表达式语句或尾表达式。



#### 4.18.4 loop 表达式

当当前 token 为 `loop` 时，解析器将其解析为 `Expr::Loop`，其内部包含一个语句块。这使得 `loop` 既可以作为循环结构，也可以作为带 `break value;` 的表达式结构使用。

#### 4.18.5 数组字面量

数组字面量形如：

Listing 53: 数组字面量示例

```
1 [1,2,3]
```

解析器在遇到左方括号后，调用 `parse_expression_list` 解析表达式列表，直到遇到右方括号为止。

#### 4.18.6 元组与括号表达式

圆括号结构需要区分元组表达式和普通括号表达式。当前解析逻辑是：遇到左括号后，先解析第一个表达式；如果后续出现逗号，则认为这是元组表达式；否则认为这是普通括号表达式。

例如：

Listing 54: 元组表达式与括号表达式示例

```
1 (1,2)
2 (1)
3 ()
```

其中，`(1,2)` 被解析为元组表达式，`(1)` 被解析为普通表达式，`()` 被解析为空元组表达式。

### 4.19 token 匹配辅助机制

为了简化 token 匹配逻辑，代码中定义了 `SimpleTokenKind` 枚举，用于表示不含关键字参数的普通 token 类型。

Listing 55: SimpleTokenKind 枚举

```
1 #[derive(Debug, Clone, Copy)]
2 enum SimpleTokenKind {
3     Identifier,
4     Number,
5     Assign,
6     .....
7 }
```

由于 `TokenKind::Keyword` 中还包含具体关键字，普通 token 和关键字 token 的匹配逻辑有所不同。代码中通过 `match_simple` 和 `match_keyword` 分别处理普通 token 和关键字 token。

Listing 56: 普通 token 与关键字匹配函数

```

1 fn match_simple(&mut self, expected: SimpleTokenKind) -> bool {
2     if self.check_simple(expected) {
3         self.advance();
4         true
5     } else {
6         false
7     }
8 }
9
10 fn match_keyword(&mut self, keyword: Keyword) -> bool {
11     if matches!(self.peak_kind(), Some(TokenKind::Keyword(actual)) if *actual ==
12         keyword) {
13         self.advance();
14         true
15     } else {
16         false
17     }
18 }

```

对于必须出现的 token，则使用 `expect_simple` 和 `expect_keyword`。如果匹配失败，解析器会调用 `error_here` 生成带当前位置的错误信息。

## 4.20 错误报告机制

语法错误由 `error_here` 函数统一生成。

Listing 57: 语法错误生成函数

```

1 fn error_here(&self, message: &str) -> ParseError {
2     if let Some(token) = self.tokens.get(self.current) {
3         ParseError {
4             message: format!("{message}, found '{token.lexeme}',",
5             position: token.position,
6         }
7     } else if let Some(token) = self.tokens.last() {
8         ParseError {
9             message: format!("{message}, found end of input",
10             position: token.position,
11         }
12     } else {
13         ParseError {
14             message: format!("{message}, found empty input",
15             position: Position { line: 1, column: 1 },
16         }
17     }
18 }

```

该函数根据当前解析位置生成错误。如果当前 token 存在，则错误信息中会包含实际遇到的 token 文本；如果已经到达输入末尾，则提示 `found end of input`；如果 token 列表为空，则提示 `found empty input`。这种设计提高了错误信息的可解释性。

例如，当赋值语句左侧不是合法左值时，解析器会报告 `left side of assignment is not assignable`。当函数参数缺少类型标注时，会报告 `expected ':' after parameter name`。当语句块缺少右花括号时，会报告 `expected '}' to close the block`。

## 4.21 语法分析器实现特点

综合来看，本项目语法分析器具有以下几个特点。

第一，采用手写递归下降方法，结构清晰。程序、函数、参数、类型、语句块、语句和表达式分别由不同解析函数处理，函数调用关系与语法层次基本对应，便于理解和维护。

第二，AST 设计较完整。节点类型覆盖程序、函数、绑定、块、语句、表达式和类型等主要结构，并支持序列化为 JSON，便于命令行输出和 Web 页面展示。

第三，表达式优先级处理清楚。解析器将比较、加减、乘除、一元、后缀和基本表达式拆分为多个层级，从而正确处理常见表达式的优先级和结合性。

第四，支持较多扩展语法。在最低要求之外，解析器还支持 `for`、`loop`、`break`、`continue`、引用、数组、元组、表达式块和 `if` 表达式等结构，使项目对类 Rust 语法具有更好的覆盖能力。

第五，包含基础的赋值左侧检查。虽然当前项目尚未实现完整语义分析，但解析器已经通过 `is_assignable` 限制了赋值语句左侧的合法形式，能够识别 `1=2`；这类明显非法的赋值结构。

第六，错误信息包含位置 and 实际 token。借助词法分析器提供的行列号，语法分析器能够在错误信息中指出错误发生位置 and 实际遇到的 token，有助于调试和展示。

# 5 Web 可视化系统设计

## 5.1 设计目标

为了提升词法分析器和语法分析器的可用性与展示效果，本项目在命令行运行方式之外，额外设计并实现了一个本地 Web 可视化系统。该系统的主要目标不是重新实现词法分析或语法分析逻辑，而是将已经完成的 `Easy_Lexer` 和 `Easy_Parser` 封装为一个可交互的分析服务，使用户能够在浏览器中输入类 Rust 源程序，并直观

查看 token 序列、抽象语法树以及错误信息。

相较于命令行输出，Web 可视化系统具有更好的展示性。词法分析结果可以以表格形式呈现，便于观察每个 token 的类别、文本内容和位置信息；语法分析结果可以以 JSON 或树状结构展示，便于理解源程序被解析后的层次化结构；错误信息也可以集中显示，方便用户快速判断输入程序中存在的词法或语法问题。因此，该模块既有助于开发阶段的调试，也适合课程验收和答辩时进行演示。

从系统定位上看，Web 可视化系统属于展示层和接口层。它不直接承担 token 识别和 AST 构造工作，而是作为前端页面与底层分析器之间的桥梁：前端负责接收用户输入和展示结果，后端负责调用词法分析器、语法分析器并组织返回数据。

5.2 系统结构

Web 可视化系统主要由前端页面和后端服务两部分组成。前端页面提供代码输入区域、分析触发按钮和结果展示区域；后端服务负责接收前端提交的源代码，调用词法分析和语法分析模块，并将分析结果封装为 JSON 返回。

整体结构可以概括为图 6 所示。

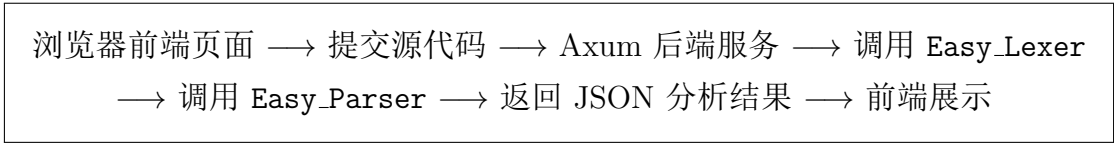


图 6: Web 可视化系统总体结构

在后端实现中，Web 服务基于 Axum 框架构建。系统启动后监听本地地址 127.0.0.1:3000，用户可以通过浏览器访问该地址进入分析页面。服务端主要设置了两个路由：根路径 / 用于返回前端页面，/api/analyze 用于接收源代码并返回分析结果。

这种设计使 Web 模块与分析器模块之间保持了较低耦合。词法分析器和语法分析器仍然可以独立用于命令行程序或测试，而 Web 服务只是对其进行调用和结果包装，不改变底层分析逻辑。

5.3 服务端路由设计

Web 服务端主要提供两个接口，如表 2 所示。

表 2: Web 服务路由设计

| 路由路径         | 请求方式 | 功能说明                                    |
|--------------|------|-----------------------------------------|
| /            | GET  | 返回前端 HTML 页面，供用户输入源代码并查看分析结果            |
| /api/analyze | POST | 接收前端提交的源代码，执行词法分析和语法分析，并返回 JSON 格式的分析结果 |

根路径 / 的作用是提供用户界面。用户访问本地服务地址后，服务端会返回内嵌的前端页面文件。前端页面中包含代码输入框、分析按钮、token 展示区域、AST 展示区域和错误信息展示区域。这样，用户无需手动执行命令程序，也可以通过浏览器完成一次完整的词法和语法分析。

/api/analyze 是 Web 系统的核心分析接口。前端将用户输入的源代码作为请求数据发送到该接口，后端收到请求后依次执行词法分析和语法分析，并将结果组织为统一的响应格式。该接口相当于把底层编译前端能力封装为一个可调用的 Web API。

5.4 请求与响应数据设计

Web 分析接口采用 JSON 作为前后端数据交换格式。前端请求中主要包含用户输入的源代码文本。后端响应中则包含 token 列表、AST、词法错误列表和语法错误信息。

请求数据的核心字段是源程序字符串。也就是说，前端只需要向后端提交一段源码文本，后端就可以完成后续分析流程。响应数据则更加丰富，主要包括以下几类信息：

- **token 列表：**保存词法分析得到的所有 token；
- **AST 结果：**保存语法分析成功后得到的抽象语法树；
- **词法错误列表：**保存词法分析阶段发现的错误；
- **语法错误信息：**保存语法分析阶段发现的错误。

其中，token 展示信息并不只是简单返回 token 的原始结构，而是面向前端显示进行了整理。每个 token 会包含所在行号、列号、原始文本、中文类别名称以及底层枚举类型。这样前端可以直接将其展示为 token 表格。例如，关键字会显示为“关键字”，标识符会显示为“标识符”，数字会显示为“数字”，运算符会显示为“算符”，括号类符号会显示为“界符”。

AST 结果在语法分析成功时返回。由于语法分析器中的 AST 节点支持序列化，因此后端可以将 Program 节点转换为 JSON 数据，并交给前端展示。如果词法分析阶段已经出现错误，则后端不会继续进行语法分析，此时 AST 为空。这种处理方式可以避免在 token 流本身不完整或不可靠的情况下继续解析，从而减少连锁错误。

## 5.5 Token 展示设计

Web 页面中的 token 展示是词法分析结果最直观的体现。后端在返回 token 时，会对底层 TokenKind 进行分类转换，将其映射为更适合用户理解的中文类别。

例如：

- Keyword 映射为“关键字”；
- Identifier 映射为“标识符”；
- Number 映射为“数字”；
- Assign、Plus、Minus 等映射为“算符”；
- 圆括号、花括号、方括号映射为“界符”；
- 分号、冒号、逗号映射为“分隔符”；
- Arrow、Dot、DotDot 映射为“特殊符号”；
- EndMarker 映射为“结束符”。

这种分类方式与课程给定的词法规则保持一致。相比直接展示 Rust 枚举名称，中文类别更便于用户理解。同时，响应中仍保留了底层枚举形式，便于在调试时查看 token 的精确类型。

127.0.0.1:3000

Summary

分析 [Ctrl+Enter]

Rust-Like Compiler

词法分析 & 语法分析可视化

源代码

```
1 fn extensions(out a: i32) -> i32 {
2   let arr: [i32; 3] = [1, 2, 3];
3   let pair: (i32, i32) = (arr[0], a);
4   return pair.0;
5 }
```

分析结果

Token 表

AST 树

错误

| #  | 位置   | Token      | 枚举   | 枚举   | 类型   |
|----|------|------------|------|------|------|
| 1  | 1:1  | fn         | 关键字  | 关键字  | 关键字  |
| 2  | 1:4  | extensions | 标识符  | 标识符  | 标识符  |
| 3  | 1:14 | {          | 界符   | 界符   | 界符   |
| 4  | 1:15 | mut        | 关键字  | 关键字  | 关键字  |
| 5  | 1:19 | a          | 标识符  | 标识符  | 标识符  |
| 6  | 1:20 | :          | 分隔符  | 分隔符  | 分隔符  |
| 7  | 1:21 | i32        | 关键字  | 关键字  | 关键字  |
| 8  | 1:24 | }          | 界符   | 界符   | 界符   |
| 9  | 1:26 | -->        | 特殊符号 | 特殊符号 | 特殊符号 |
| 10 | 1:29 | i32        | 关键字  | 关键字  | 关键字  |
| 11 | 1:33 | {          | 界符   | 界符   | 界符   |
| 12 | 2:5  | let        | 关键字  | 关键字  | 关键字  |
| 13 | 2:9  | arr        | 标识符  | 标识符  | 标识符  |
| 14 | 2:12 | :          | 分隔符  | 分隔符  | 分隔符  |
| 15 | 2:13 | [          | 界符   | 界符   | 界符   |
| 16 | 2:14 | i32        | 关键字  | 关键字  | 关键字  |
| 17 | 2:17 | :          | 分隔符  | 分隔符  | 分隔符  |
| 18 | 2:18 | 3          | 数字   | 数字   | 数字   |
| 19 | 2:19 | ]          | 界符   | 界符   | 界符   |
| 20 | 2:21 | =          | 算符   | 算符   | 算符   |
| 21 | 2:23 | {          | 界符   | 界符   | 界符   |
| 22 | 2:24 | 1          | 数字   | 数字   | 数字   |
| 23 | 2:25 | ,          | 分隔符  | 分隔符  | 分隔符  |
| 24 | 2:26 | 2          | 数字   | 数字   | 数字   |
| 25 | 2:27 | ,          | 分隔符  | 分隔符  | 分隔符  |
| 26 | 2:28 | 3          | 数字   | 数字   | 数字   |
| 27 | 2:29 | }          | 界符   | 界符   | 界符   |
| 28 | 2:30 | :          | 分隔符  | 分隔符  | 分隔符  |
| 29 | 3:5  | let        | 关键字  | 关键字  | 关键字  |
| 30 | 3:6  | .          | 分隔符  | 分隔符  | 分隔符  |
| 31 | 3:7  | 0          | 数字   | 数字   | 数字   |
| 32 | 3:8  | ;          | 分隔符  | 分隔符  | 分隔符  |

行号: 51 单词: 123

Token: 52

图 7: Web 页面中 token 展示设计

通过这种展示方式，用户可以清楚观察词法分析器如何将源代码拆分为 token。例如，输入 `if=123` 时，页面应能显示出关键字 `if`、算符 `=` 和数字 `123` 三个 token，从而验证关键字边界和 token 分割的正确性。

## 5.6 AST 展示设计

当词法分析没有错误且语法分析成功时，后端会将语法分析器生成的 AST 转换为 JSON 格式返回前端。AST 展示的目标是让用户能够观察源程序的层次化结构，例如程序由哪些函数组成，函数包含哪些参数和返回类型，函数体中有哪些语句，表达式之间具有怎样的嵌套关系。

AST 的 JSON 形式通常包含明确的 `kind` 字段。例如，顶层节点为 `Program`，函数声明节点为 `FunctionDecl`，语句节点可能为 `Let`、`Assign`、`Return`、`While` 等，表达式节点可能为 `Binary`、`Call`、`Identifier` 或 `Number` 等。通过这些节点类型，用户可以直观看到语法分析器对源程序结构的理解。

在 Web 页面中，AST 可以采用格式化 JSON 结构展示。

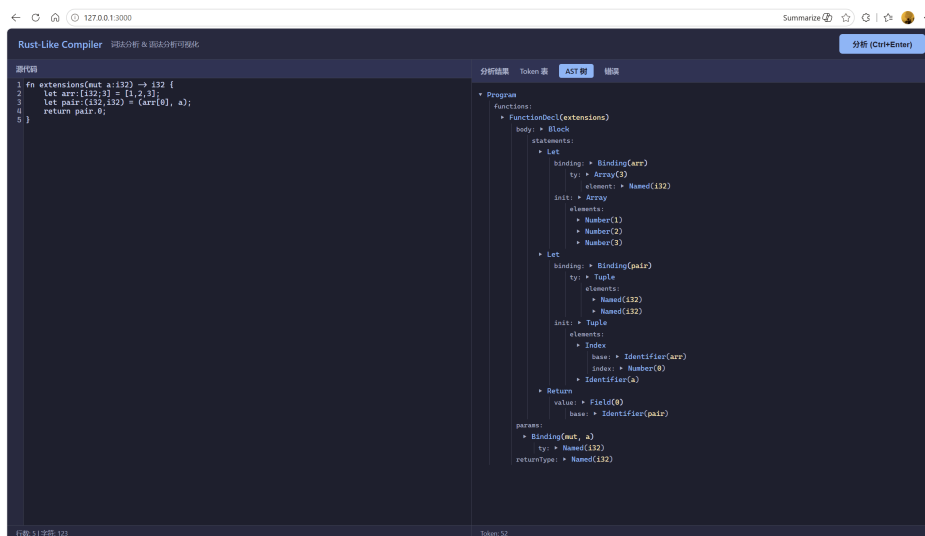


图 8: Web 页面中 AST 展示设计

AST 展示对于本项目具有重要意义。词法分析结果只能说明源程序被拆分成了哪些 token，而 AST 能进一步说明这些 token 如何组成合法的程序结构。例如，对于表达式 `a+1*2`，token 表只能显示 `a`、`+`、`1`、`*`、`2`；而 AST 可以显示乘法表达式嵌套在加法表达式右侧，从而体现表达式优先级处理的正确性。

## 5.7 错误信息展示设计

Web 系统同时展示词法错误和语法错误。二者来自不同阶段，含义也有所不同。

词法错误由词法分析器产生，主要包括无法识别的字符、未闭合块注释等问题。后端会将词法错误格式化为带有行号、列号和错误描述的字符串，并返回给前端。前端可以将这些错误集中展示在错误区域中。

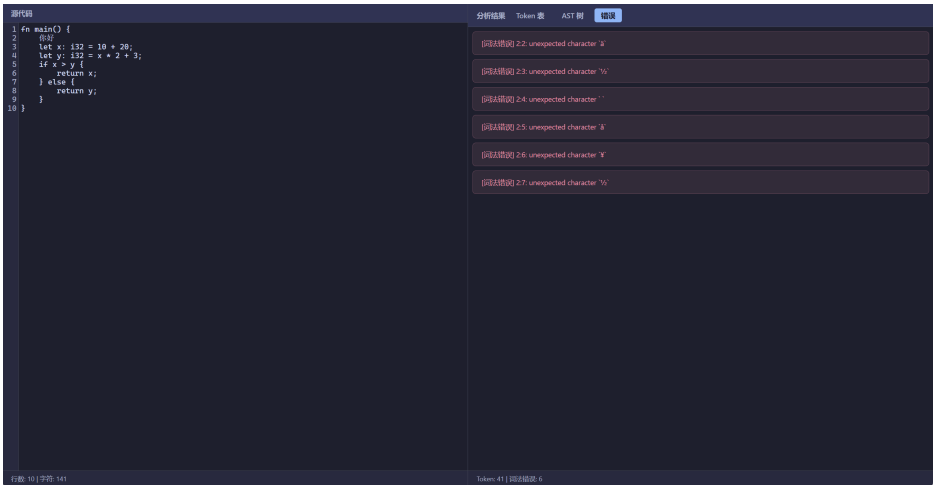


图 9: Web 页面中词法错误展示设计

语法错误由语法分析器产生，主要包括缺少分号、缺少括号、函数声明不完整、表达式不合法、赋值左侧不可赋值等问题。由于语法分析器的错误类型中也包含位置信息，因此语法错误同样可以显示具体行列号和错误原因。

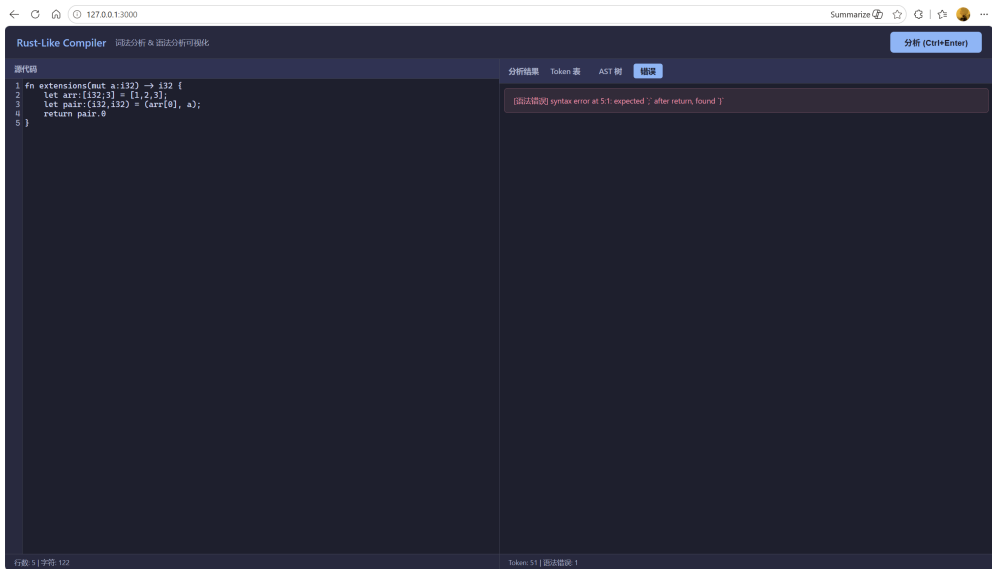


图 10: Web 页面中语法错误展示设计

本系统采用了较为清晰的错误处理顺序：如果词法分析阶段已经发现错误，则后端不会继续进行语法分析。这是合理的，因为语法分析依赖 token 流的正确性；若 token 流中已经存在词法问题，继续语法分析可能产生大量连锁错误，反而干扰用户判断真正原因。

因此，Web 页面中的错误展示逻辑可以概括为：



- 若存在词法错误，则展示词法错误，并不展示 AST；
- 若词法分析成功但语法分析失败，则展示语法错误；
- 若词法分析和语法分析均成功，则展示 token 表和 AST；
- 若没有错误，则错误区域可以为空或显示“分析成功”。

这种设计使用户能够按照编译前端的处理顺序理解错误来源：先检查字符级和 token 级问题，再检查语法结构问题。

5.8 Web 可视化系统的作用

Web 可视化系统在本项目中主要发挥两个方面的作用。

第一，它提高了系统的可操作性。用户不需要分别运行词法分析器和语法分析器命令，也不需要手动查看 JSON 文件，只需要在浏览器中输入代码即可获得完整分析结果。

第二，它验证了模块复用能力。Web 服务本身并没有重新实现词法分析和语法分析，而是直接调用 `Easy_Lexer` 和 `Easy_Parser`。这说明项目底层模块具有较好的独立性，可以被命令行程序、测试程序和 Web 服务共同复用。

6 实验测试与结果分析

6.1 测试环境

表 3: 测试环境

| 项目   | 内容                       |
|------|--------------------------|
| 操作系统 | Windows                  |
| 编程语言 | Rust                     |
| 构建工具 | Cargo                    |
| 浏览器  | Edge / Chrome            |
| 测试方式 | 单元测试、集成测试、命令行测试、Web 页面测试 |

6.2 词法分析测试

代码中为词法分析器设计了多个单元测试，用于验证关键词法功能是否符合预期。这些测试不仅验证了基本 token 识别，也覆盖了课程中特别强调的边界情况。

### 6.2.1 关键字后缀测试

测试 `keyword_suffix_stays_identifier` 用于验证 `if123` 不会被错误拆分，而是作为一个完整标识符识别。

Listing 58: 关键字后缀测试

```

1 #[test]
2 fn keyword_suffix_stays_identifier() {
3     let result = lex("if123");
4     assert!(result.errors.is_empty());
5     assert_eq!(result.tokens.len(), 1);
6     assert_eq!(result.tokens[0].kind, TokenKind::Identifier);
7     assert_eq!(result.tokens[0].lexeme, "if123");
8 }

```

该测试说明词法分析器采用的是“先完整读取标识符，再判断是否为关键字”的策略，而不是简单的关键字前缀匹配。

### 6.2.2 关键字、赋值号和整数拆分测试

测试 `keyword_operator_number_are_split` 用于验证 `if=123` 能够被正确拆分为关键字、赋值号和数字。

Listing 59: 关键字、赋值号和整数拆分测试

```

1 #[test]
2 fn keyword_operator_number_are_split() {
3     let result = lex("if=123");
4     assert!(result.errors.is_empty());
5     assert_eq!(
6         result
7             .tokens
8             .iter()
9             .map(|token| &token.kind)
10            .collect::<Vec<_>>(),
11         vec![
12             &TokenKind::Keyword(Keyword::If),
13             &TokenKind::Assign,
14             &TokenKind::Number,
15         ]
16     );
17 }

```

该测试对应课程要求中的特殊样例，验证了词法分析器在关键字边界和不同 token 分界上的正确性。

### 6.2.3 注释跳过测试

测试 `comments_are_skipped` 用于验证块注释和单行注释不会生成 token。

Listing 60: 注释跳过测试

```

1  #[test]
2  fn comments_are_skipped() {
3      let result = lex("let /* block */ mut // line\n a");
4      assert!(result.errors.is_empty());
5      assert_eq!(
6          result
7              .tokens
8              .iter()
9              .map(|token| token.lexeme.as_str())
10             .collect::<Vec<_>>(),
11          vec!["let", "mut", "a"]
12      );
13 }

```

该测试表明，注释内容被正确忽略，最终输出中只保留有效 token。

### 6.2.4 最长匹配测试

测试 `longest_match_tokens_win` 用于验证复合 token 的优先识别。

Listing 61: 最长匹配测试

```

1  #[test]
2  fn longest_match_tokens_win() {
3      let result = lex("-> == >= <= != .. .");
4      assert!(result.errors.is_empty());
5      assert_eq!(
6          result
7              .tokens
8              .iter()
9              .map(|token| &token.kind)
10             .collect::<Vec<_>>(),
11          vec![
12              &TokenKind::Arrow,
13              &TokenKind::EqualEqual,
14              &TokenKind::GreaterEqual,
15              &TokenKind::LessEqual,
16              &TokenKind::NotEqual,
17              &TokenKind::DotDot,
18              &TokenKind::Dot,
19          ]
20      );
21 }

```

该测试说明 `->`、`==`、`>=`、`<=`、`!=` 和 `..` 都会优先于其单字符前缀被识别，从而满足词法分析中的最长匹配原则。

### 6.2.5 未闭合块注释测试

测试 `unterminated_block_comment_reports_error` 用于验证未闭合块注释能够被正确报告为词法错误。

Listing 62: 未闭合块注释测试

```

1 #[test]
2 fn unterminated_block_comment_reports_error() {
3     let result = lex("/* oops");
4     assert!(result.tokens.is_empty());
5     assert_eq!(result.errors.len(), 1);
6     assert!(result.errors[0].message.contains("unterminated"));
7 }

```

该测试说明词法分析器能够发现块注释缺少结束符的问题，并将其记录在错误列表中。

### 6.2.6 词法分析结果

词法分析的测试结果如下：

```

(base) PS D:\TJ_CompileTheory\Easy_Lexer> cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 4.17s
    Running unittests src\lib.rs (target\debug\deps\easy_lexer-6b890151834d3469.exe)

running 5 tests
test tests::comments_are_skipped ... ok
test tests::keyword_suffix_stays_identifier ... ok
test tests::keyword_operator_number_are_split ... ok
test tests::unterminated_block_comment_reports_error ... ok
test tests::longest_match_tokens_win ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

    Running unittests src\main.rs (target\debug\deps\My_Lexer-a3436e2c44035913.exe)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

    Running tests\file_input_output.rs (target\debug\deps\file_input_output-2f8ac4fc83c4cde8.exe)

running 1 test
test reads_from_input_file_and_writes_output_file ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.08s

   Doc-tests easy_lexer

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

```

图 11: 词法分析结果截图

## 6.3 语法分析测试

代码中为语法分析器设计了多个测试用例，用于验证最低语法要求、扩展语法、错误处理和结束符处理。

### 6.3.1 最低语法规则测试

测试 `parses_minimum_required_grammar` 覆盖了课程要求中的核心语法结构：

Listing 63: 最低语法规则测试示例

```

1  #[test]
2  fn parses_minimum_required_grammar() {
3      parse_ok(
4          r#"
5          fn base(mut a:i32) -> i32 {
6              ;
7              let mut b:i32;
8              let c=1;
9              a=a+1*2;
10             if a>0 {
11                 foo(a, c);
12             }
13             while a!=0 {
14                 a=a-1;
15             }
16             return a;
17         }
18
19         fn foo(x:i32, y:i32) {
20             x+y;
21         }
22         "#,
23     );
24 }
```

该测试包含函数声明、参数、返回类型、空语句、变量声明、变量声明赋值、赋值语句、表达式、if 结构、while 循环、函数调用和返回语句，能够较好验证课程强制要求是否被覆盖。

### 6.3.2 扩展语法测试

测试 `parses_extended_constructs` 用于验证扩展语法：

Listing 64: 扩展语法测试示例

```

1  #[test]
2  fn parses_extended_constructs() {
3      let ast = parse_ok(
4          r#"
5          fn extra(mut a:i32) -> i32 {
6              let mut arr:[i32;3]=[1,2,3];
7              let mut pair:(i32,i32)=(arr[0], a);
8              let r:&mut i32=&mut a;
9              let v={
10                  let t=pair.0;
11                  if t>0 { t } else { 0 }
12              };
13          }
14          "#,
15     );
16 }
```

```

13         for mut i in 0..a {
14             if i==2 { continue; }
15         }
16         let b=loop {
17             break v;
18         };
19         return b;
20     }
21     "#,
22 );
23     serde_json::to_string_pretty(&ast).unwrap();
24 }

```

该测试覆盖数组类型和数组字面量、元组类型和元组字段访问、可变引用、块表达式、if 表达式、for 循环、区间表达式、continue、loop 表达式和 break 表达式等内容。测试中还调用 `serde_json::to_string_pretty`，验证 AST 能够正确序列化为 JSON。

### 6.3.3 赋值目标错误测试

测试 `reports_assignment_target_errors` 用于验证非法赋值目标能够被识别：

Listing 65: 赋值目标错误测试

```

1  #[test]
2  fn reports_assignment_target_errors() {
3      let result = lex(r#"
4          fn bad() {
5              1=2;
6          }
7          "#);
8      let error = parse_tokens(&result.tokens).unwrap_err();
9      assert!(error.message.contains("not assignable"));
10 }

```

该测试说明解析器并不是简单地接受所有“表达式 = 表达式”的形式，而是会检查左侧表达式是否满足可赋值条件。

### 6.3.4 结束符测试

测试 `accepts_end_marker` 用于验证课程要求中的结束符 `#` 可以被接受：

Listing 66: 结束符测试

```

1  #[test]
2  fn accepts_end_marker() {
3      parse_ok(
4          r#"
5          fn done() {
6              return;
7          }

```

```
8
9
10
11
```

```
    #
    "#,
);
}
```

该测试对应 `parse_program` 中对 `EndMarker` 的处理逻辑。

### 6.3.5 语法分析结果

语法分析的测试结果如下：

```
(base) PS D:\TJ_CompileTheory\Easy_Parser> cargo test
Finished `test` profile [unoptimized + debuginfo] target(s) in 1.12s
Running unittests src\lib.rs (target\debug\deps\easy_parser-c6c605198951703f.exe)

running 4 tests
test tests::accepts_end_marker ... ok
test tests::parses_minimum_required_grammar ... ok
test tests::reports_assignment_target_errors ... ok
test tests::parses_extended_constructs ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Running unittests src\main.rs (target\debug\deps\My_Parser-de59123d973c0a16.exe)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Running tests\file_input_output.rs (target\debug\deps\file_input_output-e8808e28a216d5f7.exe)

running 1 test
test reads_lexer_input_file_and_produces_parser_output ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.09s

Doc-tests easy_parser

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

图 12: 语法分析结果截图

## 7 改进方向

当前项目已经完成了类 Rust 语言的词法分析和语法分析主体，能够将源程序转换为 token 序列，并进一步构造抽象语法树。但从完整编译器前端的角度来看，当前系统仍主要停留在词法和语法结构识别阶段，尚未形成完整的语义分析能力。后续可以在现有 AST 基础上，从符号表、类型检查和可变性检查三个方面继续完善。

### 7.1 增加符号表

当前语法分析器能够识别变量声明、函数参数、赋值语句和表达式等结构，但尚未系统记录程序中已经声明过的变量、函数及其作用域信息。因此，后续可以增加符号表结构，用于保存变量名、类型、是否可变、声明位置、所属作用域等信息。

符号表可以随着语法树遍历过程逐步建立。例如，在进入一个函数体或语句块时创建新的作用域，在遇到变量声明或函数参数时将其加入当前作用域，在离开语

句块时销毁对应作用域。通过这种方式，可以支持对局部变量、函数参数以及嵌套语句块中变量可见性的管理。

引入符号表后，系统可以进一步检查以下问题：

- 变量在使用前是否已经声明；
- 同一作用域中是否存在重复声明；
- 内层作用域变量是否正确覆盖外层变量；
- 赋值语句或表达式中引用的标识符是否能够在当前作用域中找到。

因此，符号表是后续语义分析的基础。没有符号表，语法分析器只能判断程序结构是否符合文法；有了符号表后，系统才能进一步判断程序中名称使用是否合理。

## 7.2 实现基础类型检查

在符号表基础上，后续可以进一步实现基础类型检查。当前项目虽然在 AST 中已经能够表示 `i32`、数组、元组、引用等类型结构，但这些类型信息主要用于语法结构展示，并没有系统参与语义判断。例如，解析器可以识别 `let a:i32;`、`let arr:[i32;3];`、`let pair:(i32,i32);` 等声明，但还不会完整检查后续赋值或表达式计算是否符合类型规则。

例如，对于函数：

Listing 67: 返回类型检查示例

```
1 fn f() -> i32 {  
2     return;  
3 }
```

语法上它可能可以被解析，但从语义上看，函数声明返回 `i32`，而 `return;` 没有返回表达式，因此应当报告返回类型不一致错误。类似地，若将数组赋值给 `i32` 变量，也应当被类型检查阶段识别为错误。

因此，类型检查可以使系统从“能否解析程序结构”进一步提升到“能否判断程序结构是否具有合理含义”。

## 7.3 实现可变性检查

Rust 语言的一个重要特点是变量默认不可变，只有显式声明为 `mut` 的变量才允许被重新赋值。当前项目在词法和语法层面已经能够识别 `mut` 关键字，并在 AST



的 `Binding` 节点中记录变量是否可变，但尚未完整检查后续是否存在对不可变变量重新赋值的情况。

后续可以在符号表中为每个变量记录其可变性属性。当语义分析器遇到赋值语句时，先根据赋值左侧的标识符在符号表中查找对应变量，再判断该变量是否被声明为 `mut`。如果变量没有 `mut` 标记，却出现在赋值语句左侧，则应当报告不可变变量赋值错误。

可变性检查不仅可以应用于普通变量，也可以进一步扩展到解引用赋值、数组元素赋值和元组字段赋值等情况。例如，`arr[0] = 1`；是否合法，既取决于 `arr` 是否可变，也取决于数组类型本身是否允许该操作。因此，可变性检查需要和符号表、类型检查结合起来实现。

综上，增加可变性检查能够更好体现类 Rust 语言的特点，也能使项目从单纯的语法分析进一步接近真实 Rust 编译器前端中的语义约束处理。

## 8 总结与展望

本项目完成了词法分析器、语法分析器和 Web 可视化展示页面，能够识别课程要求中的主要词法单元，并覆盖最低要求中的核心语法规则。同时，项目还实现了部分扩展语法，如 `else if`、`for`、`loop`、引用、数组和元组等。

从整体完成度来看，项目已经具备课程演示所需的基本形态。不过，当前系统仍以词法分析和语法分析为主，尚未实现完整语义分析。未来可以从符号表、类型检查、可变性检查、返回值检查和错误恢复等方面继续完善。

## 参考文献

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 2nd Edition, 2006.
- [2] Steve Klabnik, Carol Nichols. *The Rust Programming Language*. No Starch Press, 2019.
- [3] The Rust Project Developers. *The Rust Reference*. <https://doc.rust-lang.org/reference/>
- [4] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.
- [5] 同济大学编译原理课程组. 【Rust版】大作业1: 词法和语法分析工具设计与实现v5.0. 课程资料.